

Fatima Jinnah Women University

Department of Computer Science

Lab 02: Hands-On Git and Version Control

Lab Submission Report

Course	DevOps
Instructor	Sir Waqas Saleem
Student Name	Kiran Siddique
Registration No.	2023-BCS-052
Submission Repository	DevOps (Labs/Lab02)
Date of Submission	11 October 2026

Table of Contents

1. Introduction and Objectives 4

1.1 Environment 4

1.2 Command Reference 4

2. Getting Started: Installing Git 5

3. Task 1: Create a Private GitHub Repository 6

4. Task 2: Connect the Repository via SSH 7

5. Task 3: Configure Git Username and Email 14

6. Task 4: Explore the .git Folder 15

7. Task 5: Local Repository Management 16

8. Task 6: File Status and Staging 19

9. Task 7: Branch Creation Using the GitHub GUI 21

10. Task 8: Branch Creation and Push Using Git Bash 24

11. Task 9: Branching and Merging 26

12. Task 10: Pull Request and Branch Review 29

13. Task 11: Branch Strategy (develop and staging) 32

14. Task 12: Code Review Workflow 36

14.1 Inviting the Reviewer as a Collaborator 36

14.2 Creating the Pull Request and Assigning a Reviewer 38

14.3 Reviewer Actions: Request Changes and Reject 42

14.4 Author Actions: Fix Issues and Push New Commits 44

14.5 Approval and Merge 47

14.6 Branch Handling After Merge 50

15. Task 13: Branch Cleanup Best Practices 50

15.1 Remote Branch Deleted from the GUI 50

15.2 Remote Branch Deleted from the Terminal 51

15.3 Updating the Local Repository 51

16. Verification and Submission 52

17. Bonus: Simulated Team Collaboration 52

17.1 Creating the collab Branch 52
17.2 Adding Kiran's Name and Fun Fact 53
17.3 Second Account's Contribution 54
17.4 Merging collab into main 55
18. Exam Question Q1: Advanced Branching and Merge Verification 55
18.1 Creating the Branch 56
18.2 Making a Small Change and Committing 56
18.3 Merging Back into main 57
18.4 Verifying the Merge History 58
19. Exam Question Q2: Multi-Stage Workflow Simulation 58
19.1 Branches 58
19.2 Feature Branch and Commit 59
19.3 Merging into develop 60
19.4 Merging develop into staging 60
19.5 Merging staging into main 61
19.6 Stage Verification 62
20. Exam Question Q3: Collaboration and Conflict Resolution 62
20.1 Modifying the Shared File in Two Branches 62
20.2 Merging and the Conflict 64
20.3 Resolving the Conflict 65
20.4 Final File 66
21. Submission Structure 66

1. Introduction and Objectives

This report documents the practical work for Lab 02: Hands-On Git and Version Control. The lab covers installing Git, creating a private GitHub repository, connecting it through SSH, configuring a Git identity, inspecting the .git directory, managing local repositories, staging and committing, branching, merging, pull requests, code review, and branch cleanup. Each task lists the steps performed, the commands used with their purpose, and the screenshots that serve as evidence.

All terminal work was done on the Ubuntu virtual machine, so the prompt in each terminal screenshot is kiransiddique@ubuntu. GitHub web-interface screenshots show the browser and the terminal together where the lab requires a split view.

1.1 Environment

- Host machine: Windows PC with Windows PowerShell (used for file transfer to the VM).
- Virtual machine: Ubuntu Server running in VMware Workstation, user kiransiddique, hostname ubuntu.
- GitHub account: KiranSiddique. A second GitHub account (kiransiddique406-source) was used as the reviewer in Task 12.
- Git version used: 2.53.0 (verified with git --version).
- Repositories: DevOps_Lab2_Practice (private, practice work) and DevOps (submission, Labs/Lab02).

1.2 Command Reference

Command	Purpose
sudo apt install git -y	Installs Git on Ubuntu
git --version	Confirms the installed Git version
ssh-keygen -t ed25519 -C "email"	Generates a new SSH key pair
cat ~/.ssh/id_ed25519.pub	Displays the public key (never the private key)
ssh -T git@github.com	Tests SSH authentication with GitHub
git clone	Copies a remote repository to the local machine
git config --global user.name	Sets the author name for commits
git config --global user.email	Sets the author email for commits
git config --list	Lists all configured Git settings
ls -a / ls -a .git	Shows hidden files and the .git folder contents
rm -rf .git	Deletes the repository metadata (turns it into a plain folder)

Command	Purpose
git init	Creates a new empty repository in the current folder
git add	Stages a file for the next commit
git commit -m "msg"	Records staged changes with a message
git status	Shows the state of the working tree and staging area
git remote add origin	Links the local repository to GitHub
git branch -M main	Renames the current branch to main
git push -u origin main	Uploads main and sets it to track origin/main
git checkout -b	Creates a new branch and switches to it
git checkout	Switches to an existing branch
git merge	Merges another branch into the current branch
git pull origin	Fetches and merges a remote branch into the current branch
git branch / git branch -a	Lists local branches (and remote branches with -a)
git branch -d	Deletes a merged local branch
git push origin --delete	Deletes a branch on the remote
git log --oneline --graph --all	Shows commit history as a graph

2. Getting Started: Installing Git

Git was installed on the Ubuntu VM with the package manager. The terminal shows that Git was already the newest version and that git --version reports 2.53.0.

```
sudo apt install git -y
git --version
```

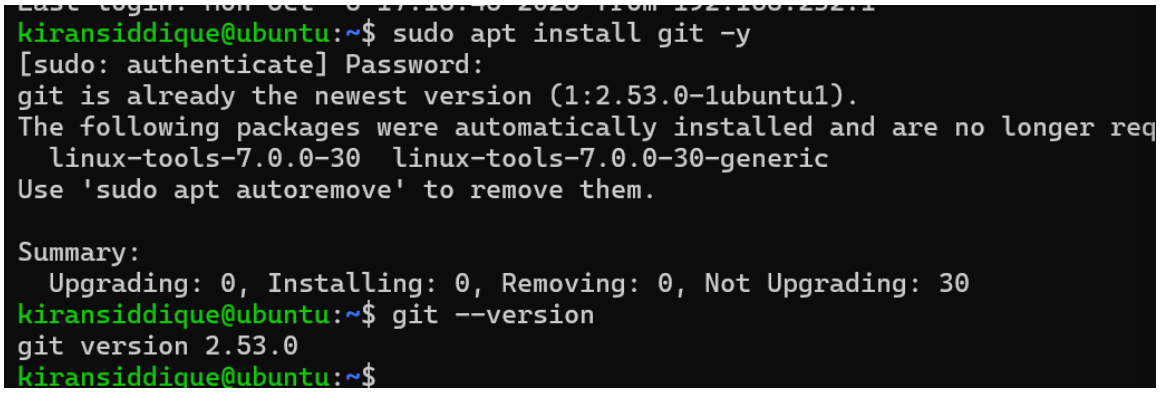


Figure 1: Git installation check in the Ubuntu terminal

```
kiransiddique@ubuntu:~$ sudo apt update
[sudo: authenticate] Password:
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:5 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [726 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [559 kB]
Get:7 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [98.4 kB]
Get:8 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [320 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.7 kB]
Get:10 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [201 kB]
Get:11 http://archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Components [1,060 B]
Get:12 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [198 kB]
Get:13 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [55.7 kB]
Fetched 2,617 kB in 12s (224 kB/s)
21 packages can be upgraded. Run 'apt list --upgradable' to see them.
kiransiddique@ubuntu:~$ sudo apt install git -y
git is already the newest version (1:2.53.0-1ubuntu1).
git set to manually installed.
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-30                linux-main-modules-zfs-7.0.0-30-generic  linux-tools-7.0.0-30-generic
  linux-headers-7.0.0-30-generic        linux-modules-7.0.0-30-generic
  linux-image-unsigned-7.0.0-30-generic  linux-tools-7.0.0-30
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 21
kiransiddique@ubuntu:~$ git --version
git version 2.53.0
kiransiddique@ubuntu:~$
```

Figure 2: Git installation and version check

3. Task 1: Create a Private GitHub Repository

- Opened GitHub and clicked New repository.
- Entered the repository name DevOps_Lab2_Practice.
- Selected Private visibility and created the repository.
- Verified on the repository page that the Private badge is shown. The page also offers Add collaborators, which was used later in Task 12.

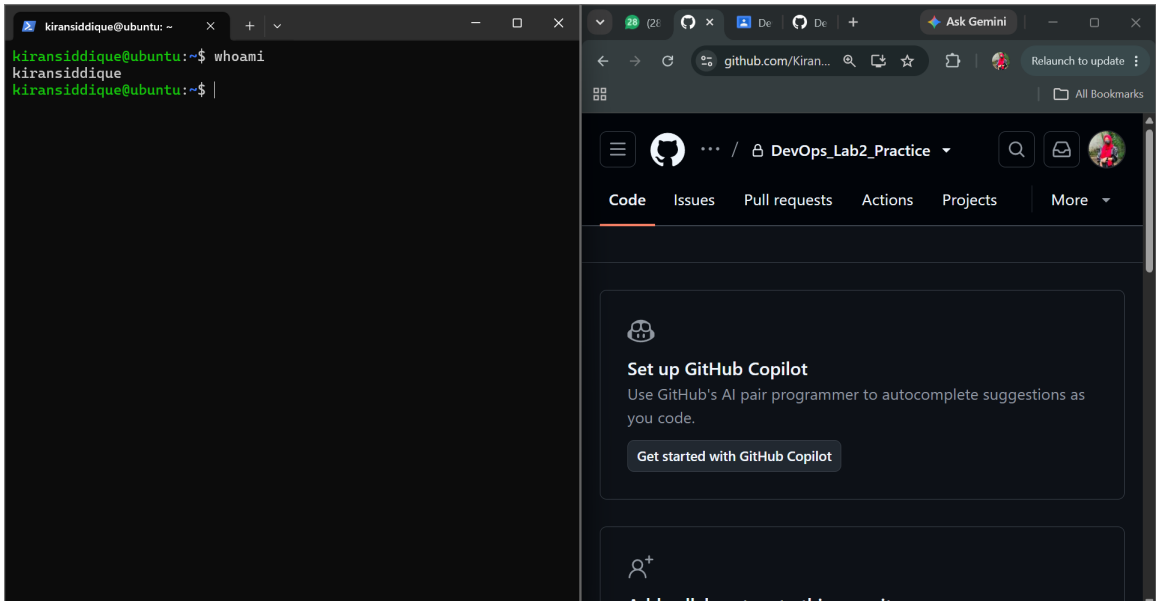


Figure 3: Private repository DevOps_Lab2_Practice

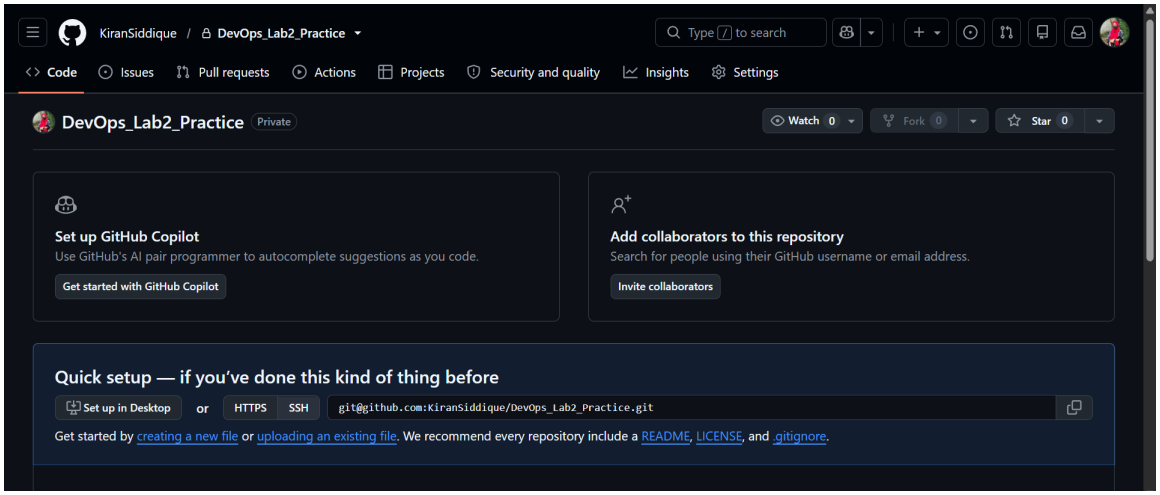


Figure 4: Repository home page after creation, showing the Private badge and setup options

4. Task 2: Connect the Repository via SSH

An Ed25519 SSH key pair was generated on the VM. The private key stays on the VM and only the public key is given to GitHub, so Git can authenticate without a password.

```
ssh-keygen -t ed25519 -C "your_email@example.com"
cat ~/.ssh/id_ed25519.pub
git clone git@github.com:KiranSiddique/DevOps_Lab2_Practice.git
```

- Ran ssh-keygen with the Ed25519 algorithm and the university email as the key comment. The first attempt accepted a wrong file name (yes), so the command was run again and the key was saved as ~/.ssh/id_ed25519.
- Displayed the public key with cat ~/.ssh/id_ed25519.pub. Only the public key was shown; the private key was never displayed.
- Opened GitHub > Settings > SSH and GPG keys. The list was empty before the key was added.
- Clicked New SSH key, entered the title Ubuntu VM, selected key type Authentication Key, pasted the public key, and clicked Add SSH key.
- GitHub asked for identity verification (Confirm access) before saving the key.
- Cloned the repository over SSH.

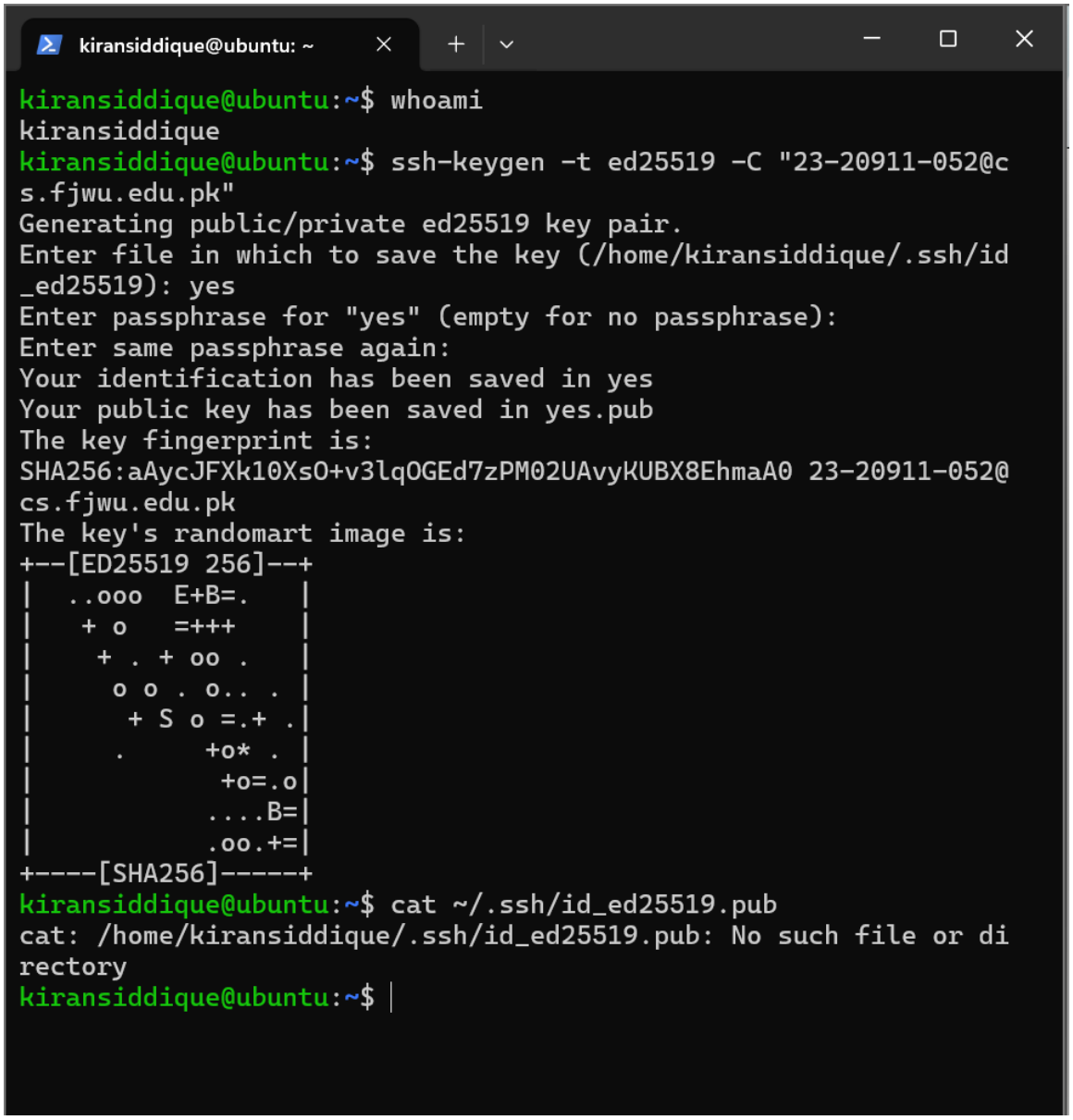


Figure 5: ssh-keygen command in the terminal (first attempt)

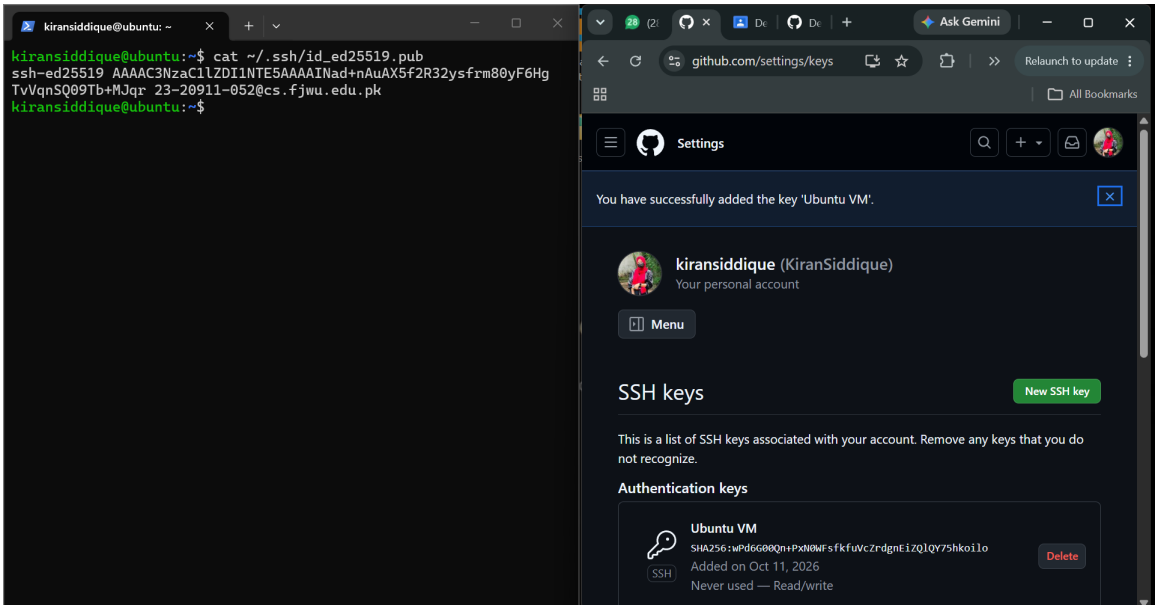


Figure 6: Public key displayed in the terminal and shown on GitHub SSH keys page

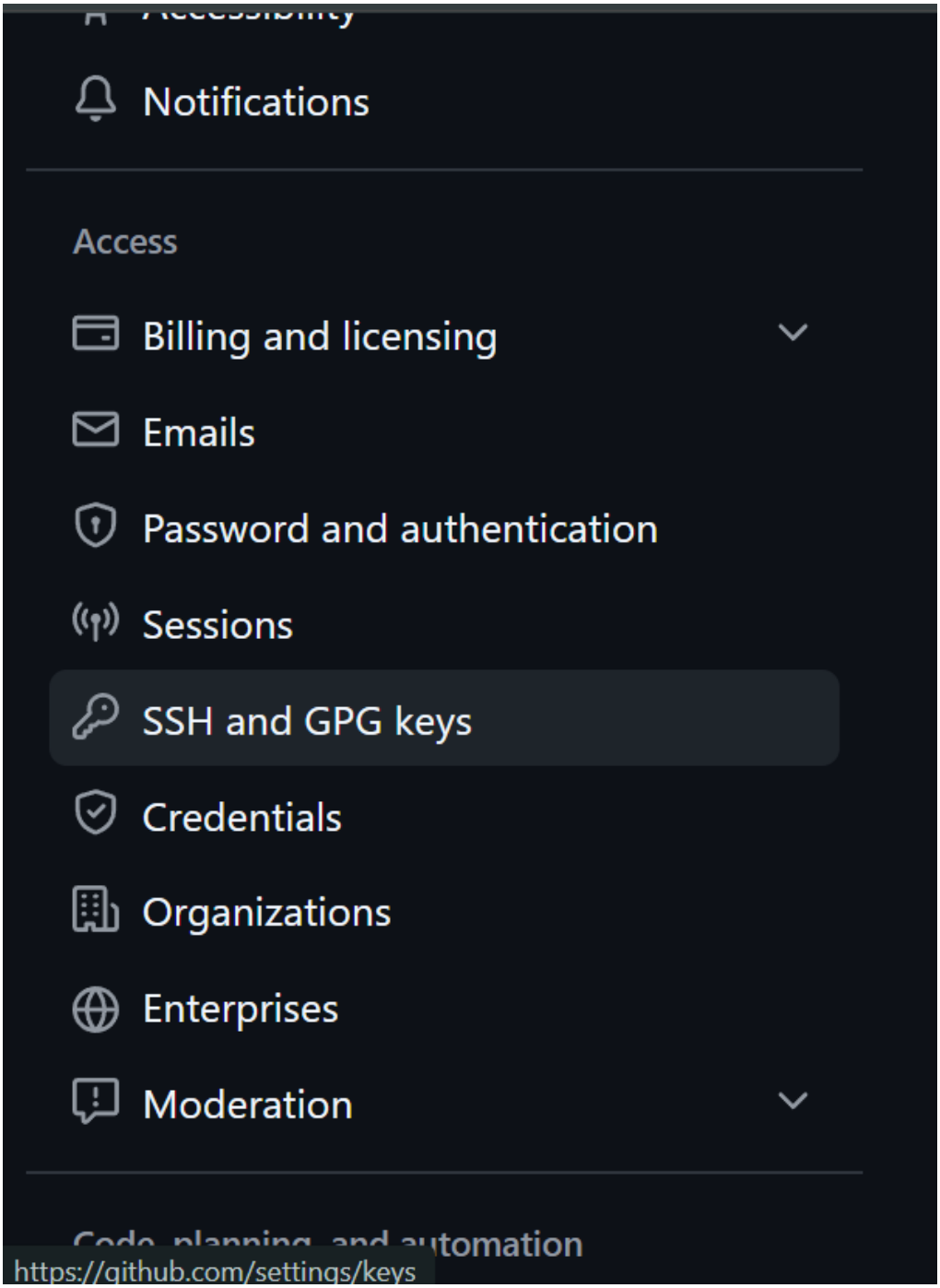


Figure 7: GitHub Settings sidebar, opening SSH and GPG keys

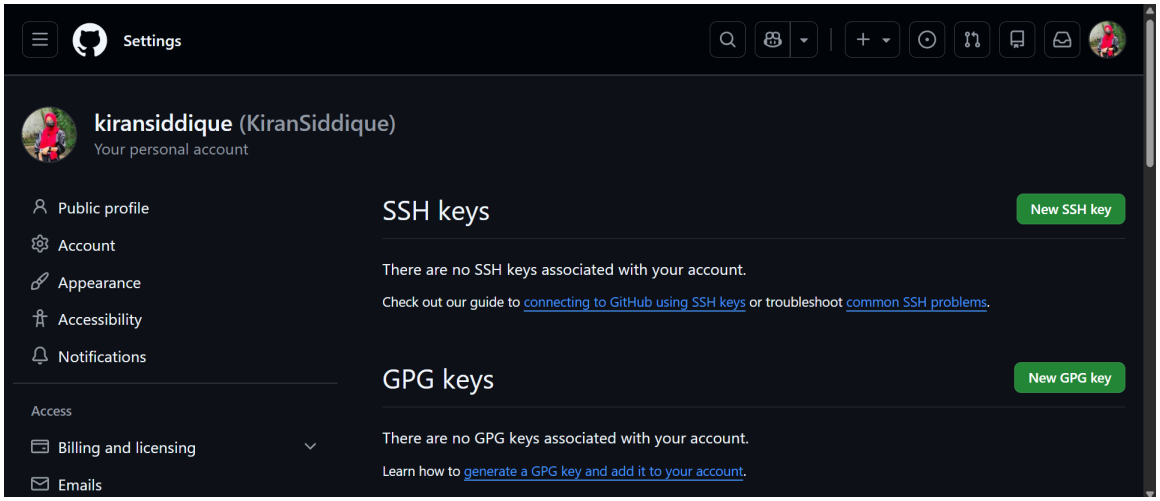


Figure 8: SSH keys page before the key is added (no keys)

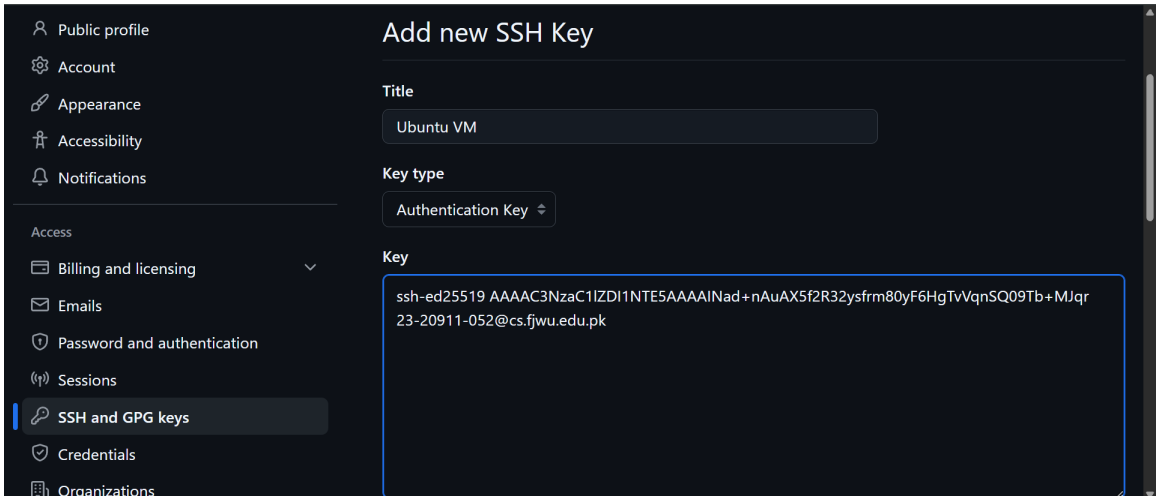


Figure 9: Add new SSH Key form with the title Ubuntu VM

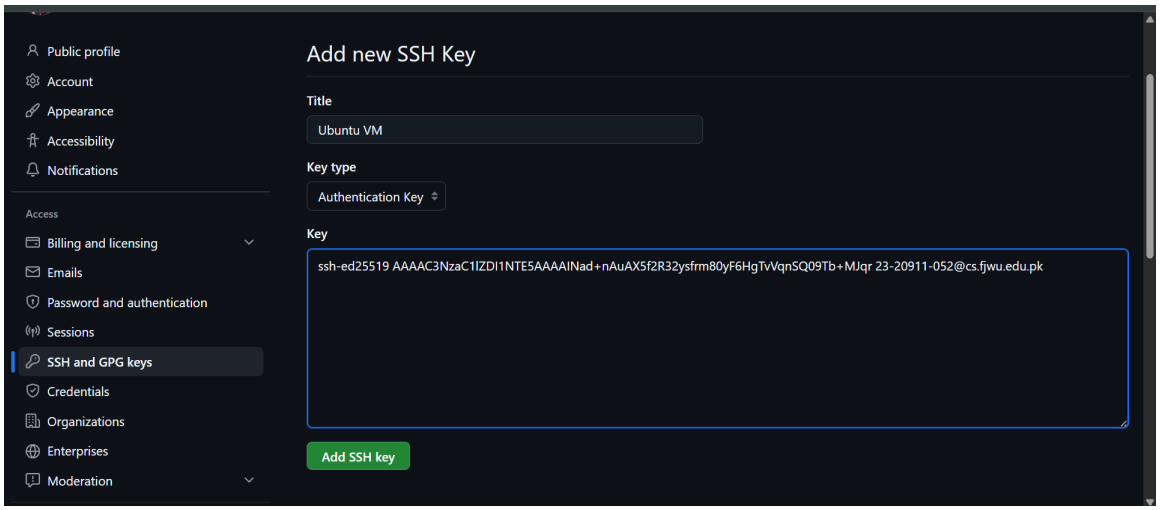


Figure 10: Public key pasted into the Key field

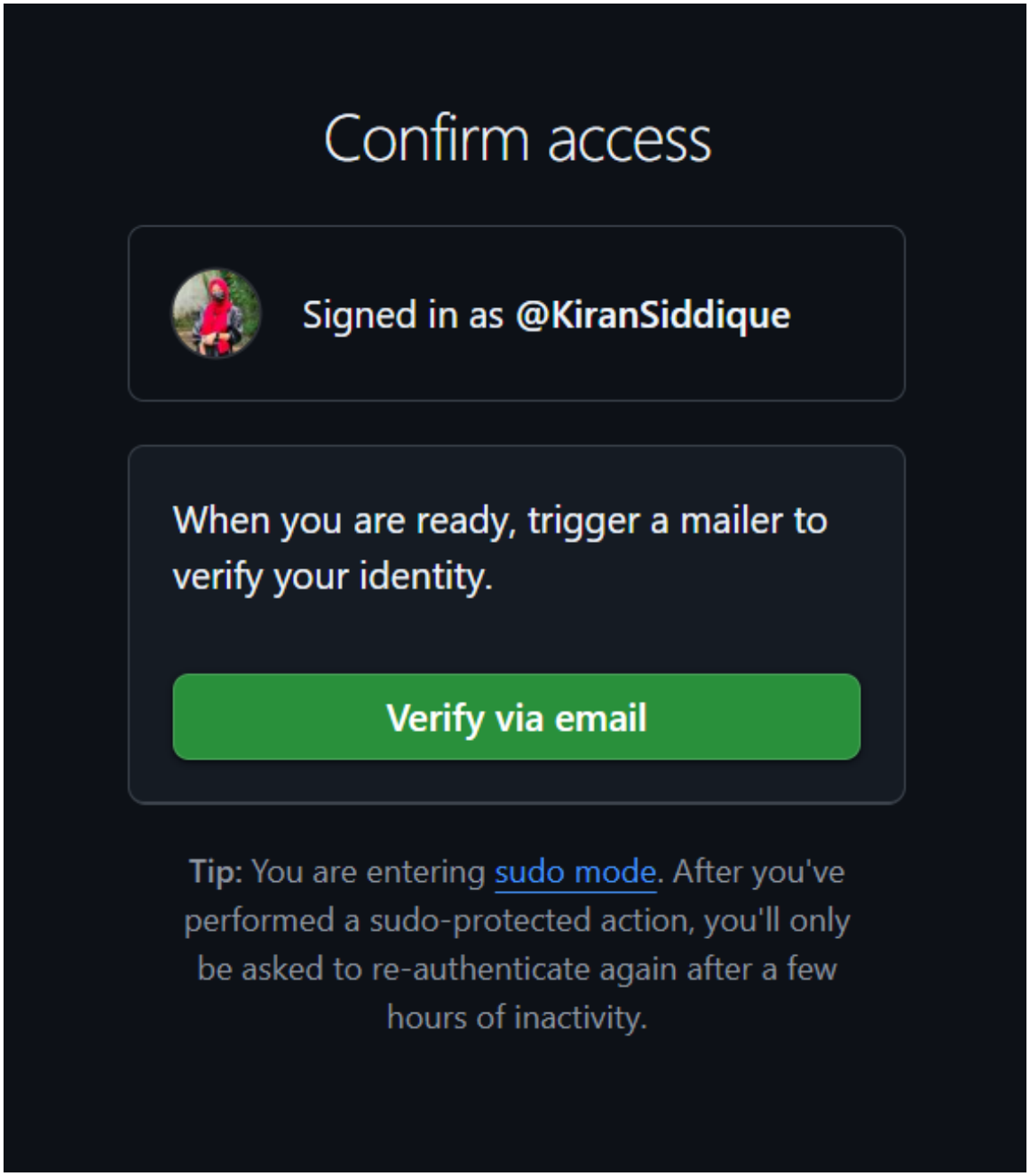


Figure 11: GitHub Confirm access page before saving the key

```
kiransiddique@ubuntu:~$ ssh-keygen -t ed25519 -C "23-20911-052@c
s.fjwu.edu.pk"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/kiransiddique/.ssh/id
_ed25519):
Enter passphrase for "/home/kiransiddique/.ssh/id_ed25519" (empt
y for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kiransiddique/.ssh/i
d_ed25519
Your public key has been saved in /home/kiransiddique/.ssh/id_ed
25519.pub
The key fingerprint is:
SHA256:wPd6G00Qn+PxN0WFsfkfuVcZrdgnEiZQlQY75hkoilo 23-20911-052@
cs.fjwu.edu.pk
The key's randomart image is:
+--[ED25519 256]--+
|        ..+o...o+|
|     .   o +o..+.|
|      + o B.B o.o|
|   . . + + O B o=|
|  E .   S + = ===|
| o       . o . +*|
|.       . o .  .o|
|         . o    .|
|          .      |
+----[SHA256]-----+
kiransiddique@ubuntu:~$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAINad+nAuAX5f2R32ysfrm80yF6Hg
TvVqnSQ09Tb+MJqr 23-20911-052@cs.fjwu.edu.pk
kiransiddique@ubuntu:~$
```

Figure 12: SSH key generation and public key display

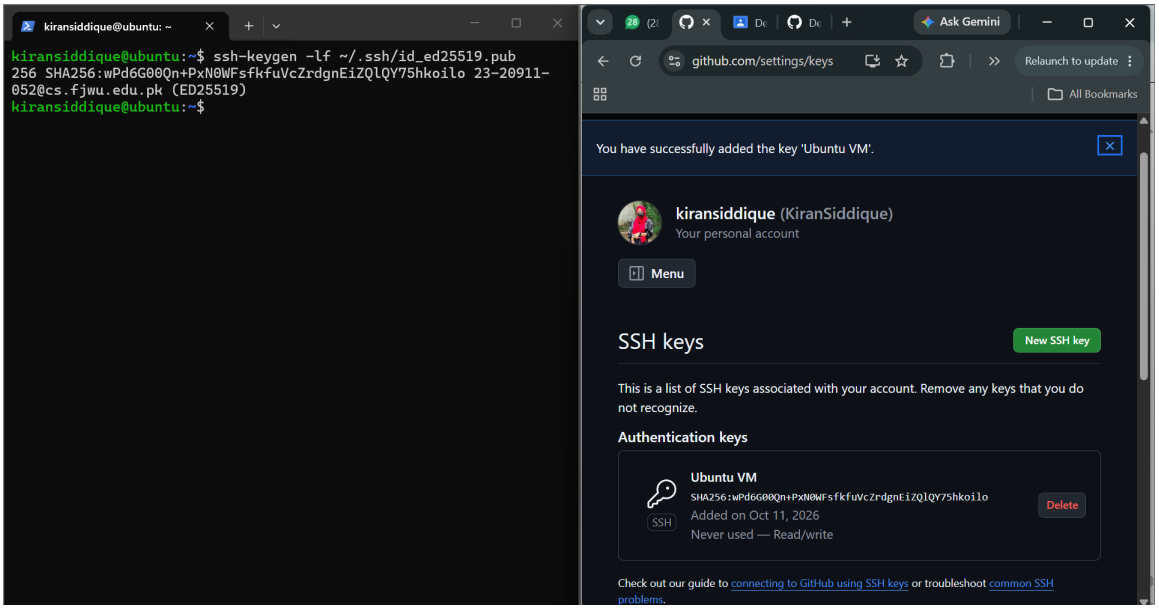


Figure 13: Public key added to GitHub SSH keys (Ubuntu VM listed)

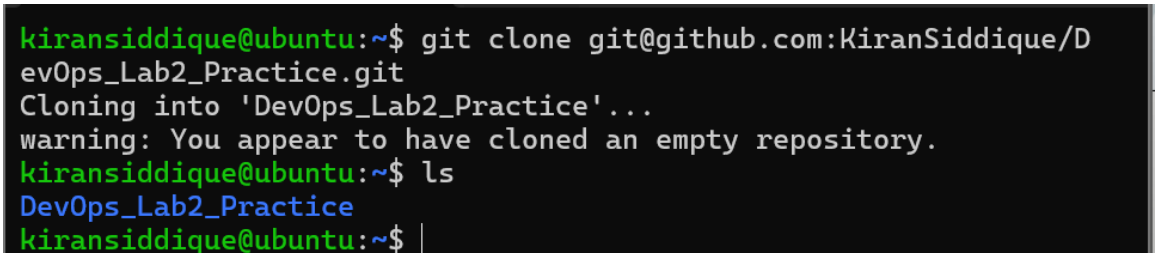


Figure 14: Repository cloned using SSH

5. Task 3: Configure Git Username and Email

Git stores the author name and email in each commit. Setting them globally links every commit to the student's GitHub identity.

```
cd ~/DevOps_Lab2_Practice
git config --global user.name "Kiran Siddique"
git config --global user.email "23-20911-052@cs.fjwu.edu.pk"
git config --list
```

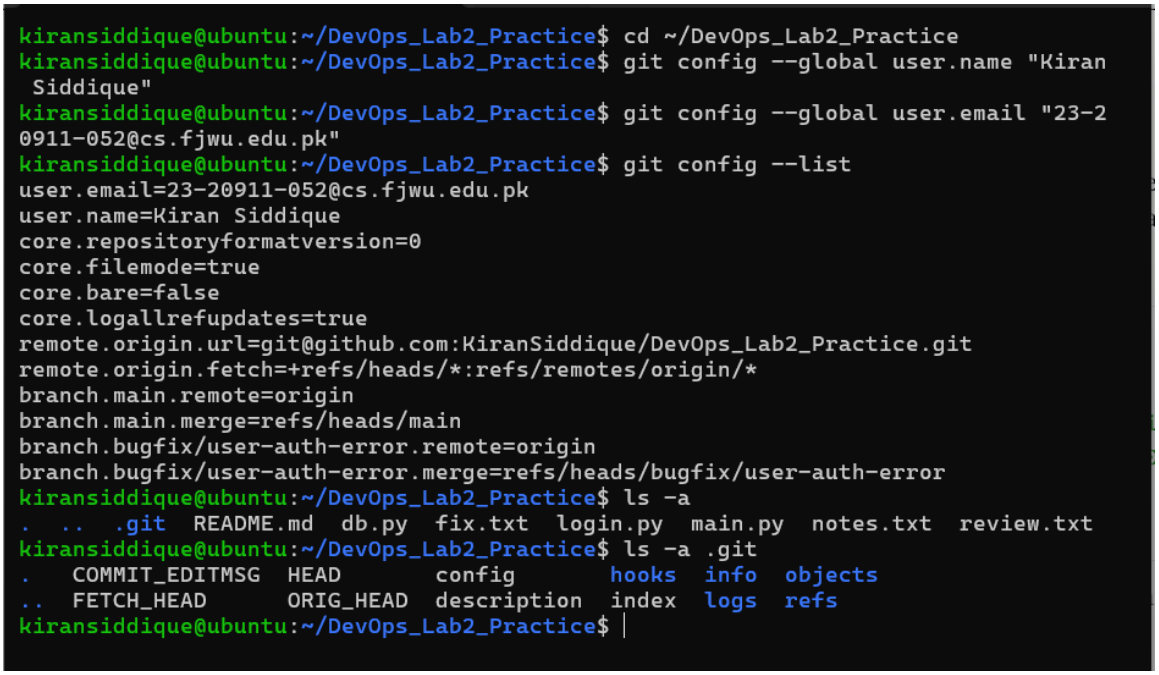


Figure 15: Setting the global Git username and email

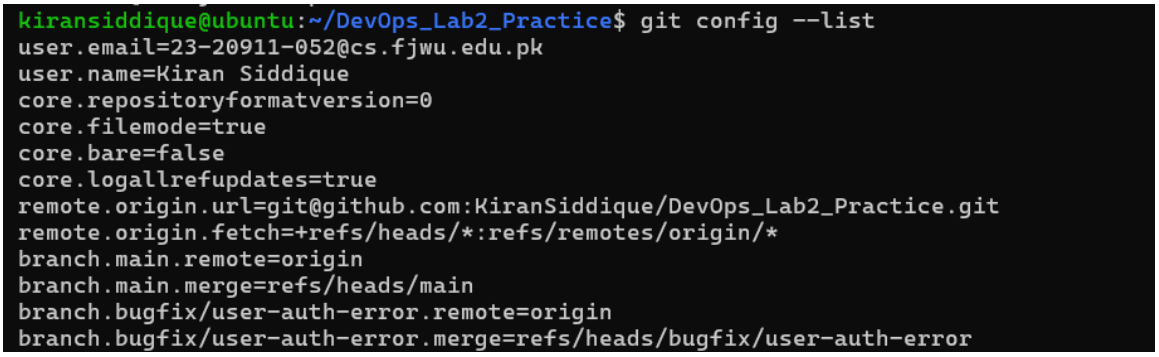


Figure 16: Output of git config --list

6. Task 4: Explore the .git Folder

The repository was entered and hidden files were listed. The .git directory stores the history, branches, configuration and object database. It must not be edited by hand.

```
cd ~/DevOps_Lab2_Practice
ls -a
ls -a .git
```

- COMMIT_EDITMSG: last commit message.
- HEAD: the branch currently checked out.
- config: repository settings (remote URL, branch tracking).
- objects and refs: stored commits, branches and tags.
- logs, index, hooks, info: history records, staging area, hook scripts and metadata.

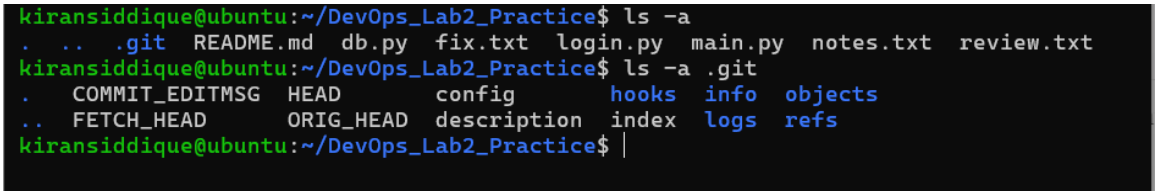


Figure 17: Hidden files and the contents of the .git directory

7. Task 5: Local Repository Management

The .git folder was deleted on purpose to show what happens without repository metadata. The repository was then re-created with git init and linked to GitHub.

- Deleted the .git folder with rm -rf .git.
- Ran git status. Git returned fatal: not a git repository, because the folder no longer has a .git directory and so Git cannot find any history or configuration.
- Re-initialized the repository with git init, which creates a new empty .git directory.
- Created README.md with the text Lab02 Git Practice, staged it, and committed it with the message Initial commit.
- Added the GitHub remote, renamed the branch to main, and pushed with upstream tracking.
- Opened the repository on GitHub to confirm that the push succeeded.

```
rm -rf .git
git status
git init
echo "# Lab02 Git Practice" >> README.md
git add README.md
git commit -m "Initial commit"
git remote add origin git@github.com:KiranSiddique/DevOps_Lab2_Practice
git branch -M main
git push -u origin main
```

```
kiransiddique@ubuntu:~$ git clone git@github.com:KiranSiddique/D
evOps_Lab2_Practice.git
Cloning into 'DevOps_Lab2_Practice'...
warning: You appear to have cloned an empty repository.
kiransiddique@ubuntu:~$ ls
DevOps_Lab2_Practice
kiransiddique@ubuntu:~$ cd ~/DevOps_Lab2_Practice
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ pwd
/home/kiransiddique/DevOps_Lab2_Practice
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ rm -rf .git
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git status
fatal: not a git repository (or any of the parent directories):
.git
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 18: Deleting the .git folder and the resulting git status error

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ ls -a
.  ..
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git init
hint: Using 'master' as the name for the initial branch. This de
fault branch name
hint: will change to "main" in Git 3.0. To configure the initial
 branch name
hint: to use in all of your new repositories, which will suppres
s this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'tru
nk' and
hint: 'development'. The just-created branch can be renamed via
this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBr
anchName false"
Initialized empty Git repository in /home/kiransiddique/DevOps_L
ab2_Practice/.git/
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 19: Re-initializing the repository with git init

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ echo "# Lab02 Git P
ractice" >> README.md
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git add README.md
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Init
ial commit"
[master (root-commit) d8607bc] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 20: Creating README.md, staging it, and the first commit

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ echo "# Lab02 Git P
ractice" >> README.md
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git add README.md
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Init
ial commit"
[master (root-commit) d8607bc] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 21: Initial commit recorded

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git remote add orig
in git@github.com:KiranSiddique/DevOps_Lab2_Practice.git
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git branch -M main
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git push -u origin
main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 243 bytes | 81.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:KiranSiddique/DevOps_Lab2_Practice.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 22: Adding the GitHub remote, renaming the branch to main, and pushing

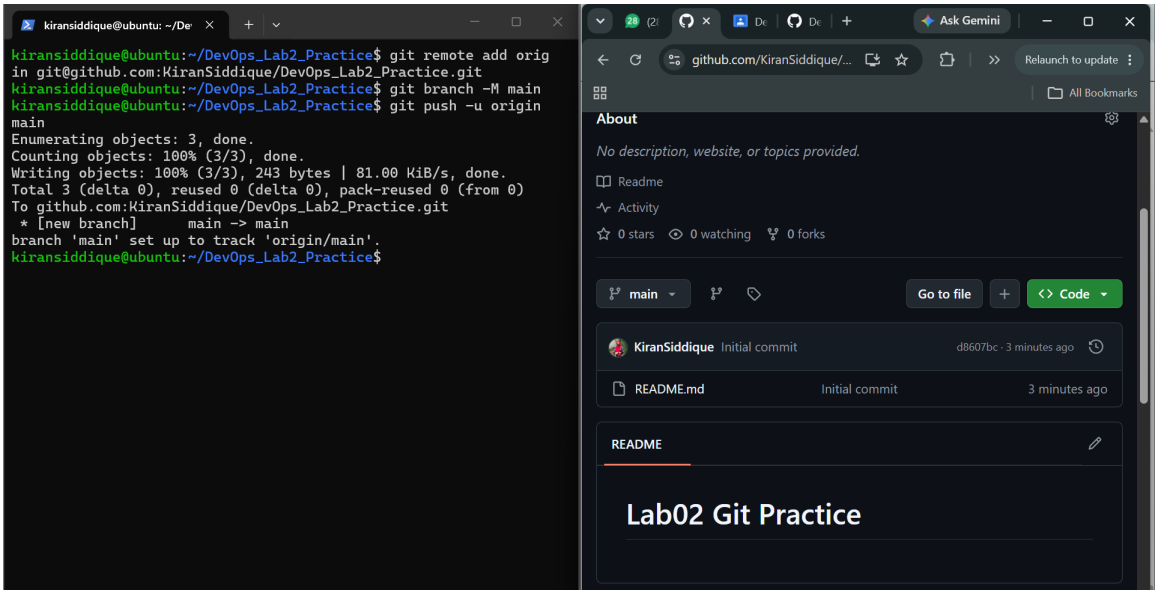


Figure 23: Pushing main to origin with upstream tracking

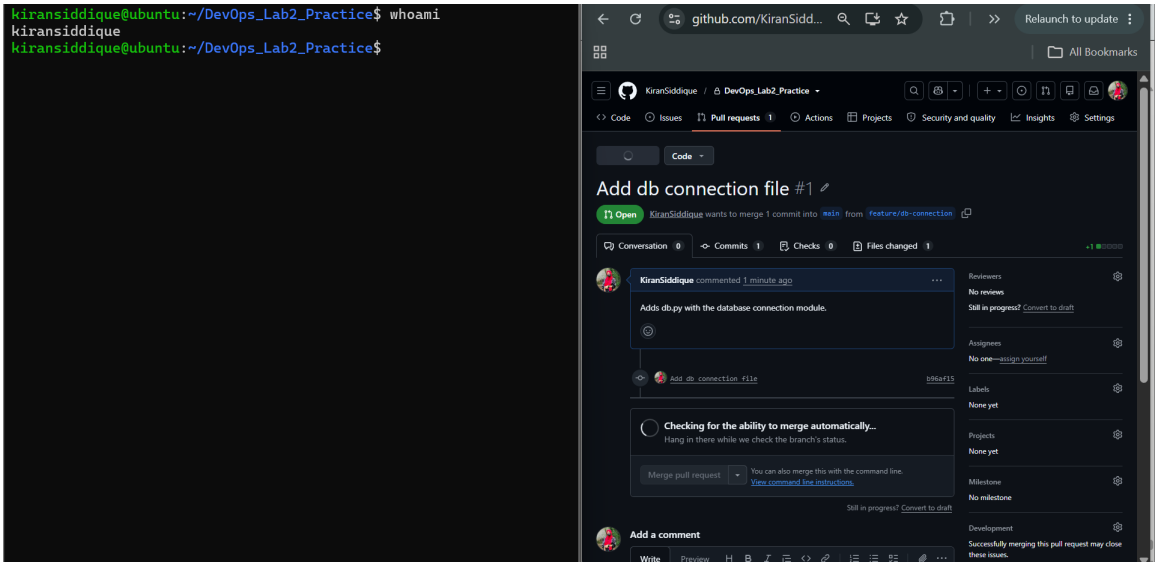


Figure 24: Terminal (whoami) and the repository page on GitHub after the push

8. Task 6: File Status and Staging

A new file was created, its state was checked, and it was staged and committed. The same process was then repeated after editing the file.

```
echo "Adding some test notes" >> notes.txt
git status
git add notes.txt
git commit -m "Add notes.txt"
```

git status showed notes.txt as an untracked file (red). After git add it moved to the staging area, and git commit recorded it.

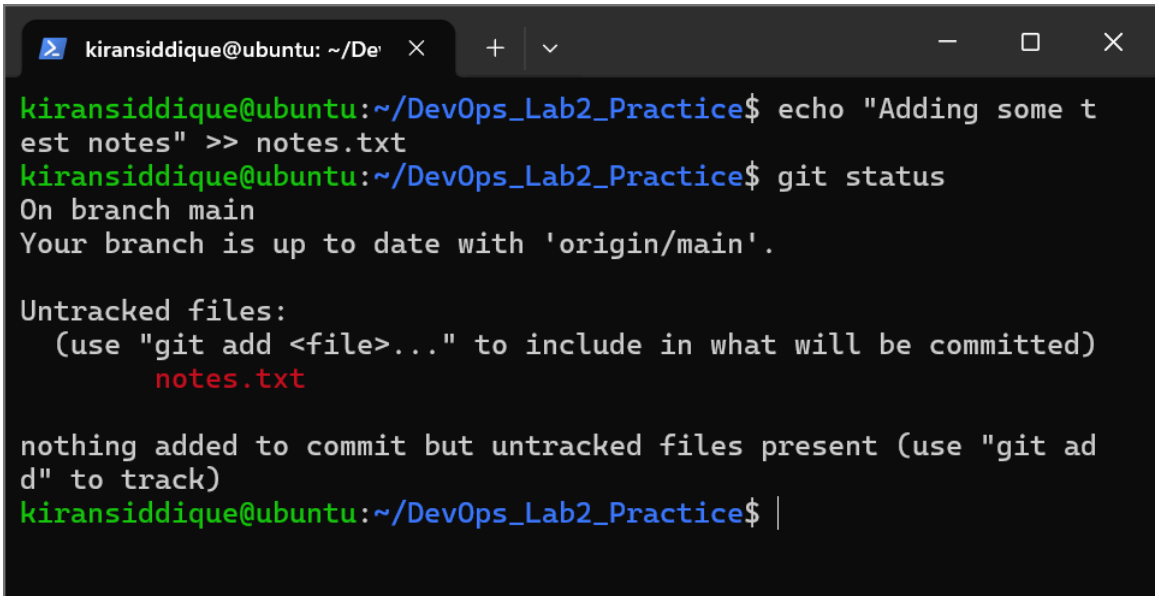


Figure 25: Creating notes.txt and running git status (untracked file)

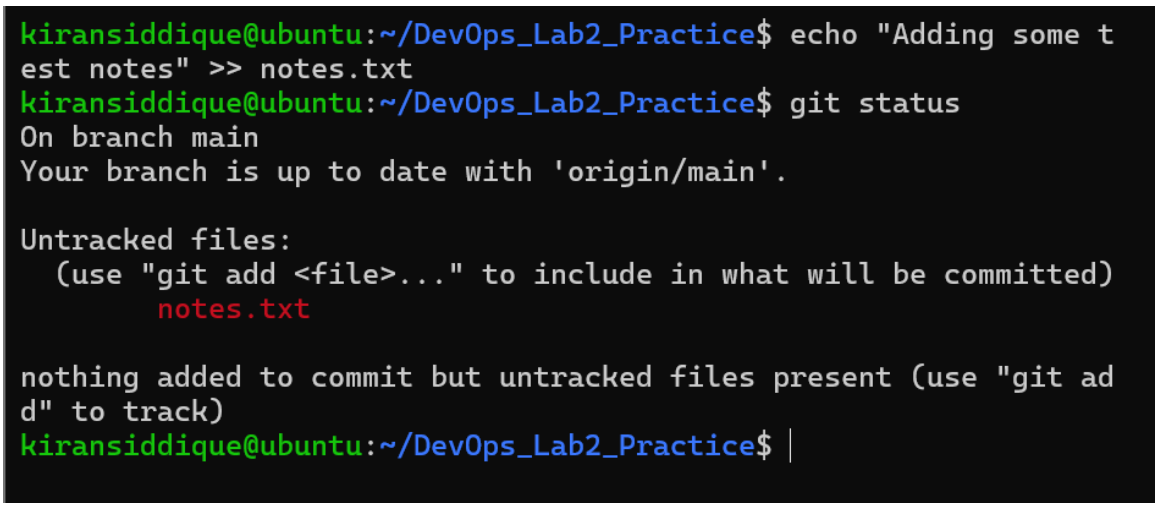


Figure 26: git status output for the new notes.txt

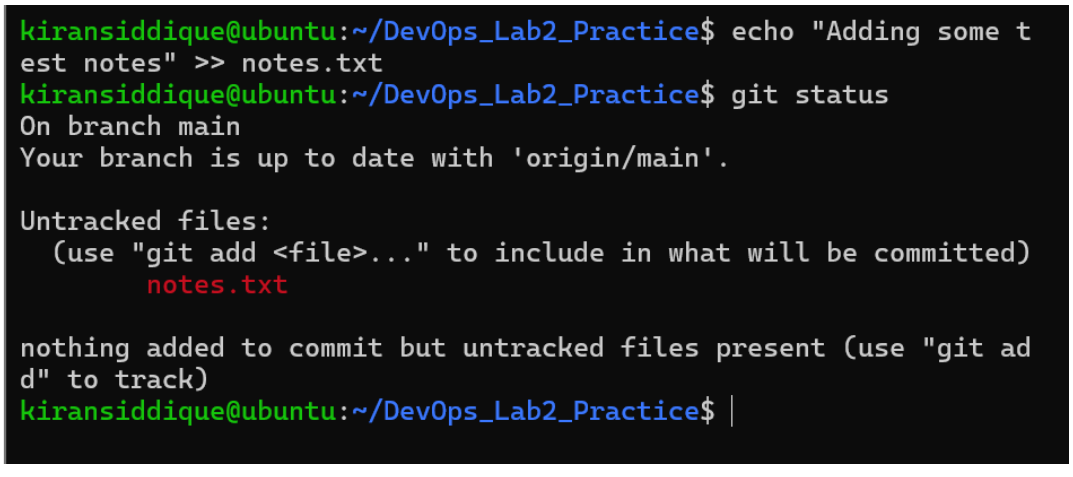


Figure 27: git status showing an untracked file

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git add notes.txt
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Add
notes.txt"
[main 5626cf0] Add notes.txt
 1 file changed, 1 insertion(+)
 create mode 100644 notes.txt
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 28: Staging and committing notes.txt

The file was then edited with a second line of text, and the same status and commit steps were repeated.

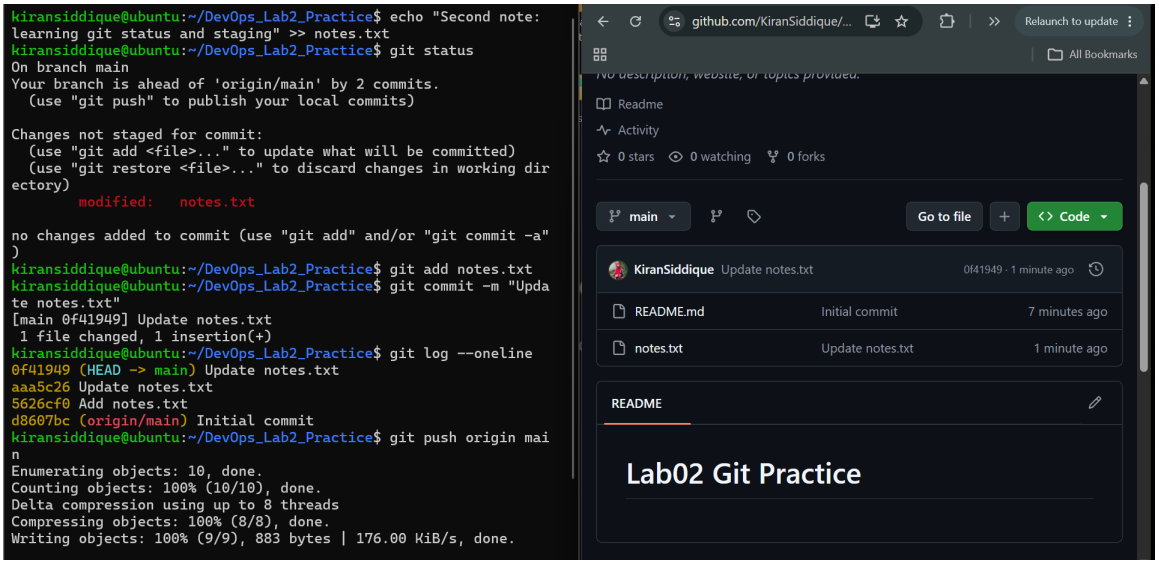


Figure 29: Second edit of notes.txt, followed by git status

9. Task 7: Branch Creation Using the GitHub GUI

- On GitHub, opened the branch selector and typed bugfix/user-auth-error.
- Selected the option Create branch bugfix/user-auth-error from main.
- On the VM, pulled the branch to sync the local repository.
- Listed local and remote branches to confirm that the branch exists on both sides. The SSH connection was also checked with ssh -T git@github.com, which greets the user without giving shell access.

```
git pull origin bugfix/user-auth-error
git branch -a
ssh -T git@github.com
```

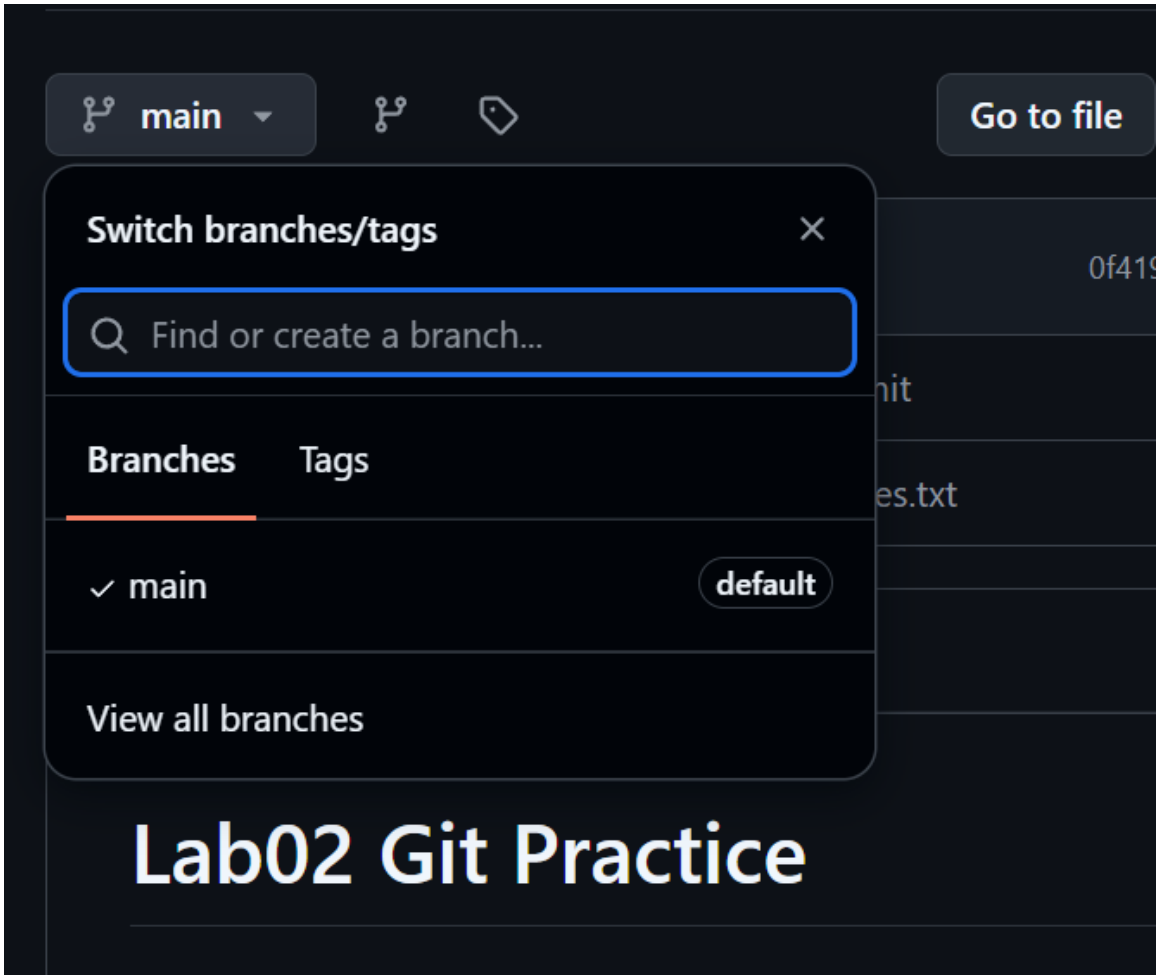


Figure 30: Branch selector on GitHub, showing the new branch option

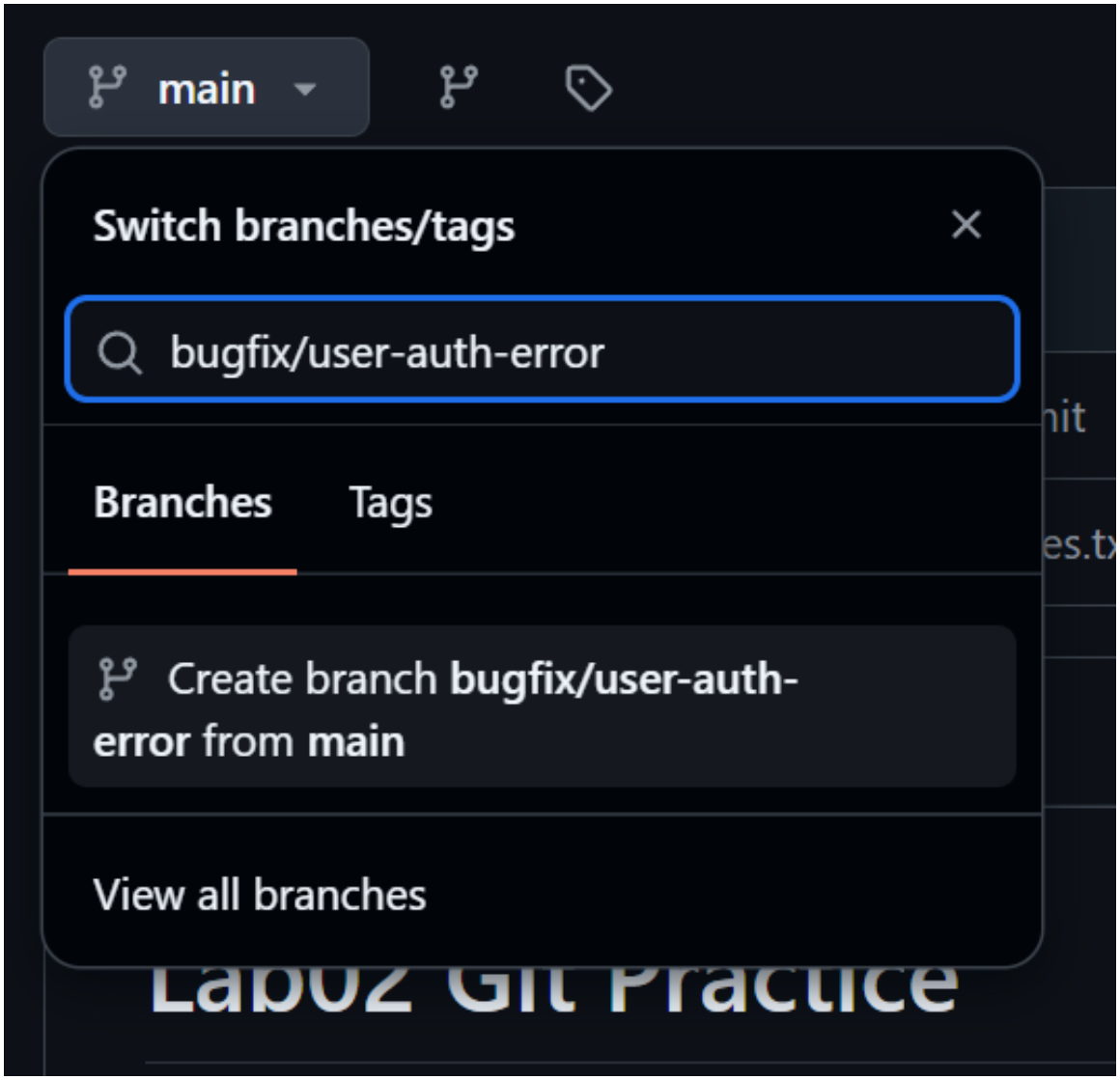


Figure 31: Typing bugfix/user-auth-error and creating it from main

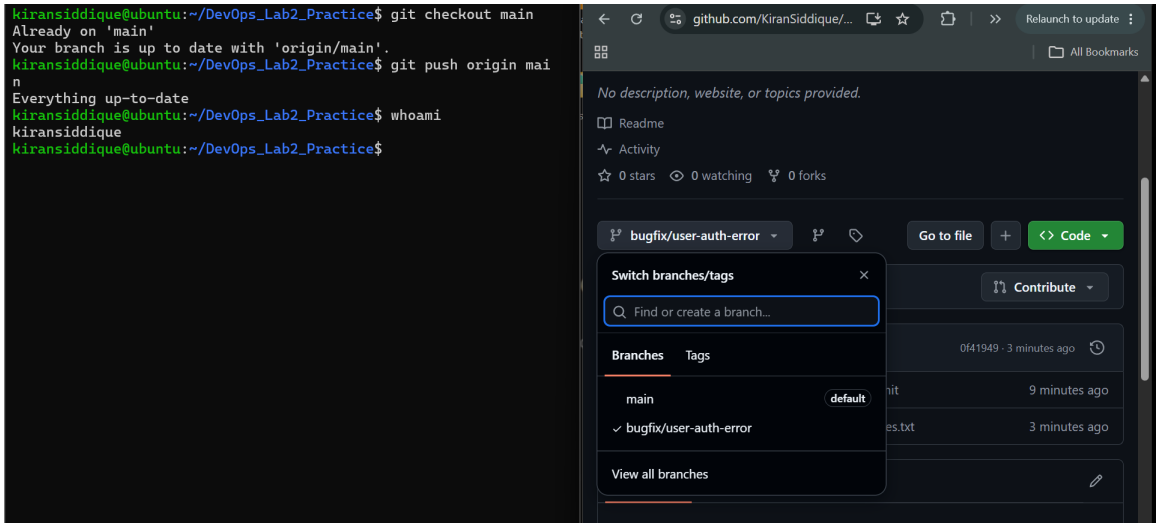


Figure 32: Branch bugfix/user-auth-error created in the GitHub GUI

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git pull origin bug
fix/user-auth-error
From github.com:KiranSiddique/DevOps_Lab2_Practice
 * branch            bugfix/user-auth-error -> FETCH_HEAD
 * [new branch]      bugfix/user-auth-error -> origin/bugfix/use
r-auth-error
Already up to date.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout bugfix
/user-auth-error
branch 'bugfix/user-auth-error' set up to track 'origin/bugfix/u
ser-auth-error'.
Switched to a new branch 'bugfix/user-auth-error'
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git branch
* bugfix/user-auth-error
  main
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 33: Local branch list after pulling the branch

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ cd ~/DevOps_Lab2_Pr
actice
git status
git branch -a
ssh -T git@github.com
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
  bugfix/user-auth-error
  feature-1
  feature/db-connection
* main
  remotes/origin/bugfix/user-auth-error
  remotes/origin/feature-1
  remotes/origin/feature/db-connection
  remotes/origin/main
Hi KiranSiddique! You've successfully authenticated, but GitHub
does not provide shell access.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 34: git branch -a and ssh -T test in the terminal

10. Task 8: Branch Creation and Push Using Git Bash

- Created and switched to feature/db-connection.
- Added the database connection file (db.py) and committed it.
- Pushed the branch to origin. GitHub then displayed the Compare and pull request button.

```
git checkout -b feature/db-connection
git push origin feature/db-connection
```

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout -b fea
ture/db-connection
Switched to a new branch 'feature/db-connection'
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ echo "# Database co
nnection module" > db.py
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git add db.py
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Add
db connection file"
[feature/db-connection b96af15] Add db connection file
 1 file changed, 1 insertion(+)
 create mode 100644 db.py
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git push origin fea
ture/db-connection
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 351 bytes | 175.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/db-connection' on Git
Hub by visiting:
remote:      https://github.com/KiranSiddique/DevOps_Lab2_Practi
ce/pull/new/feature/db-connection
remote:
To github.com:KiranSiddique/DevOps_Lab2_Practice.git
 * [new branch]      feature/db-connection -> feature/db-connect
ion
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 35: Branch feature/db-connection created and pushed

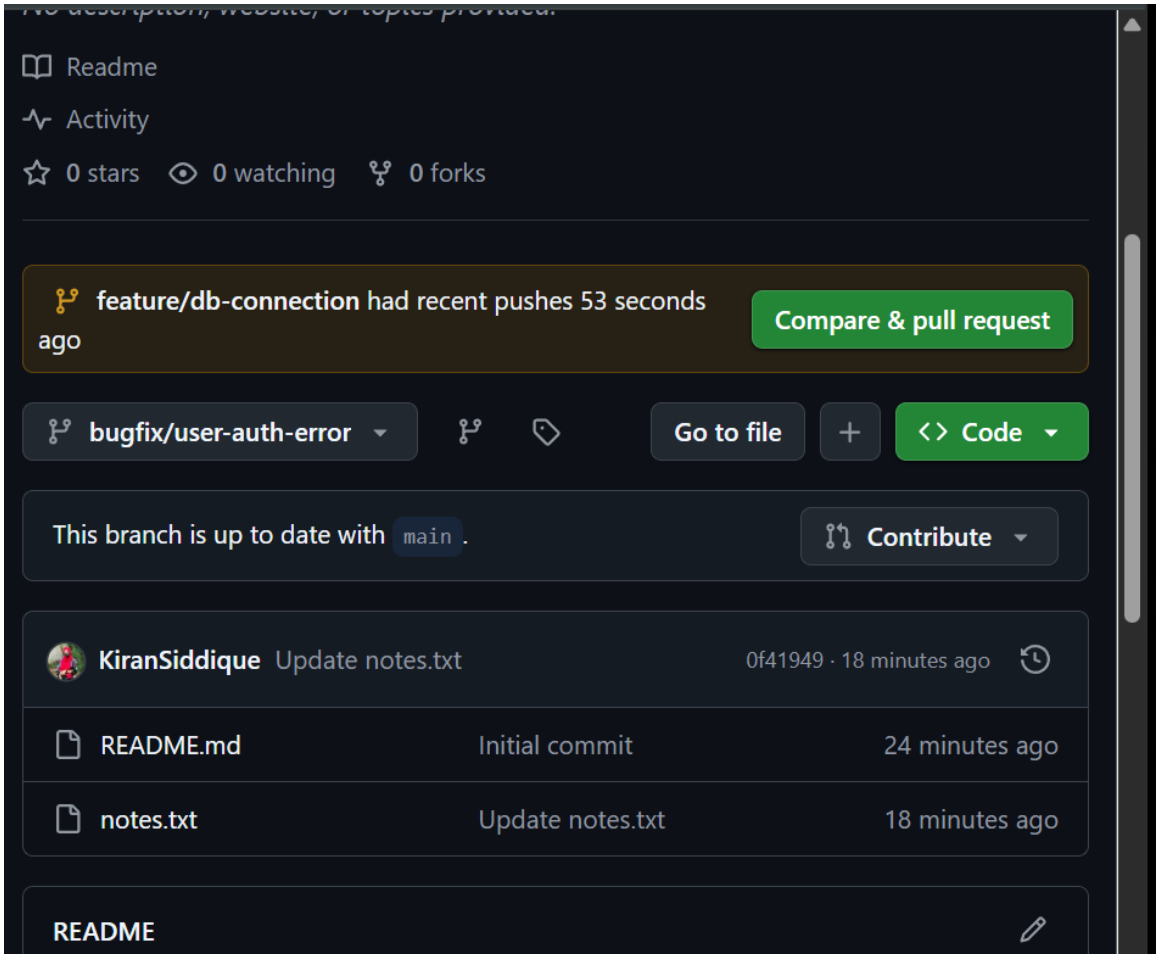


Figure 36: GitHub notice: feature/db-connection had recent pushes, with Compare & pull request

11. Task 9: Branching and Merging

A feature branch was used to add a function to main.py, and the branch was then merged locally into main. The lab notes that direct local merges into main are risky on a team, so Pull Requests are used for the later tasks.

```
git checkout -b feature-1
git add main.py
git commit -m "Add new function to main.py"
git checkout main
git merge feature-1
git push origin main
git push origin feature-1
```

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout -b fea
ture-1
Switched to a new branch 'feature-1'
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git branch
  bugfix/user-auth-error
* feature-1
  feature/db-connection
  main
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 37: Creating and switching to feature-1

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ cat > main.py << 'E
OF'
def hello():
    print("Hello DevOps")

hello()
EOF
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git add main.py
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Add
main.py"
[main a213414] Add main.py
 1 file changed, 4 insertions(+)
 create mode 100644 main.py
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git push origin mai
n
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 355 bytes | 177.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:KiranSiddique/DevOps_Lab2_Practice.git
   0f41949..a213414  main -> main
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 38: Adding a function to main.py, then staging and committing it

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ cat >> main.py << '
EOF'

def add(a, b):
    return a + b
EOF
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ cat main.py
def hello():
    print("Hello DevOps")

hello()

def add(a, b):
    return a + b

def add(a, b):
    return a + b
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git add main.py
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Add
new function to main.py"
[feature-1 0de085a] Add new function to main.py
 1 file changed, 3 insertions(+)
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 39: Commit of the new function on feature-1

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git merge feature-1

Updating a213414..0de085a
Fast-forward
 main.py | 6 ++++++
 1 file changed, 6 insertions(+)
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git log --oneline
0de085a (HEAD -> main, feature-1) Add new function to main.py
765dad5 Add new function to main.py
a213414 (origin/main) Add main.py
0f41949 (origin/bugfix/user-auth-error, bugfix/user-auth-error)
Update notes.txt
aaa5c26 Update notes.txt
5626cf0 Add notes.txt
d8607bc Initial commit
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 40: Merging feature-1 into main

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git push origin mai
n
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 698 bytes | 232.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To github.com:KiranSiddique/DevOps_Lab2_Practice.git
   a213414..0de085a  main -> main
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git push origin fea
ture-1
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature-1' on GitHub by visit
ing:
remote:      https://github.com/KiranSiddique/DevOps_Lab2_Practi
ce/pull/new/feature-1
remote:
To github.com:KiranSiddique/DevOps_Lab2_Practice.git
 * [new branch]      feature-1 -> feature-1
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 41: Pushing main and feature-1 to origin

12. Task 10: Pull Request and Branch Review

- On GitHub, opened a Pull Request with the base set to main and the compare branch set to feature/db-connection.
- Gave the Pull Request the title Add db connection file, checked that GitHub reports it can be merged automatically, and created it.
- Reviewed the Pull Request and clicked Merge pull request, then confirmed the merge.
- After the merge, GitHub showed that the branch can be safely deleted. Clicked Delete branch.

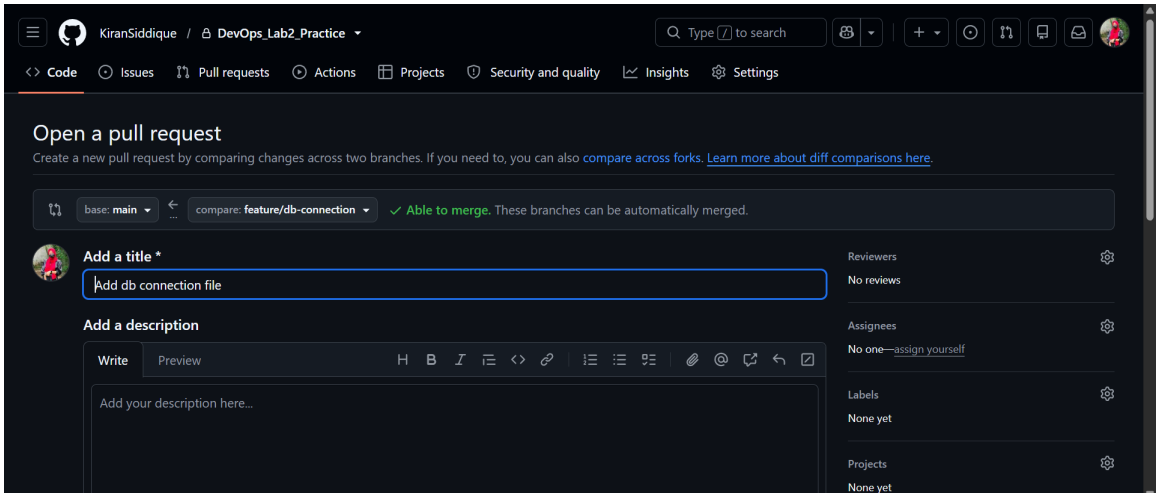


Figure 42: Open a pull request page with base main and compare feature/db-connection

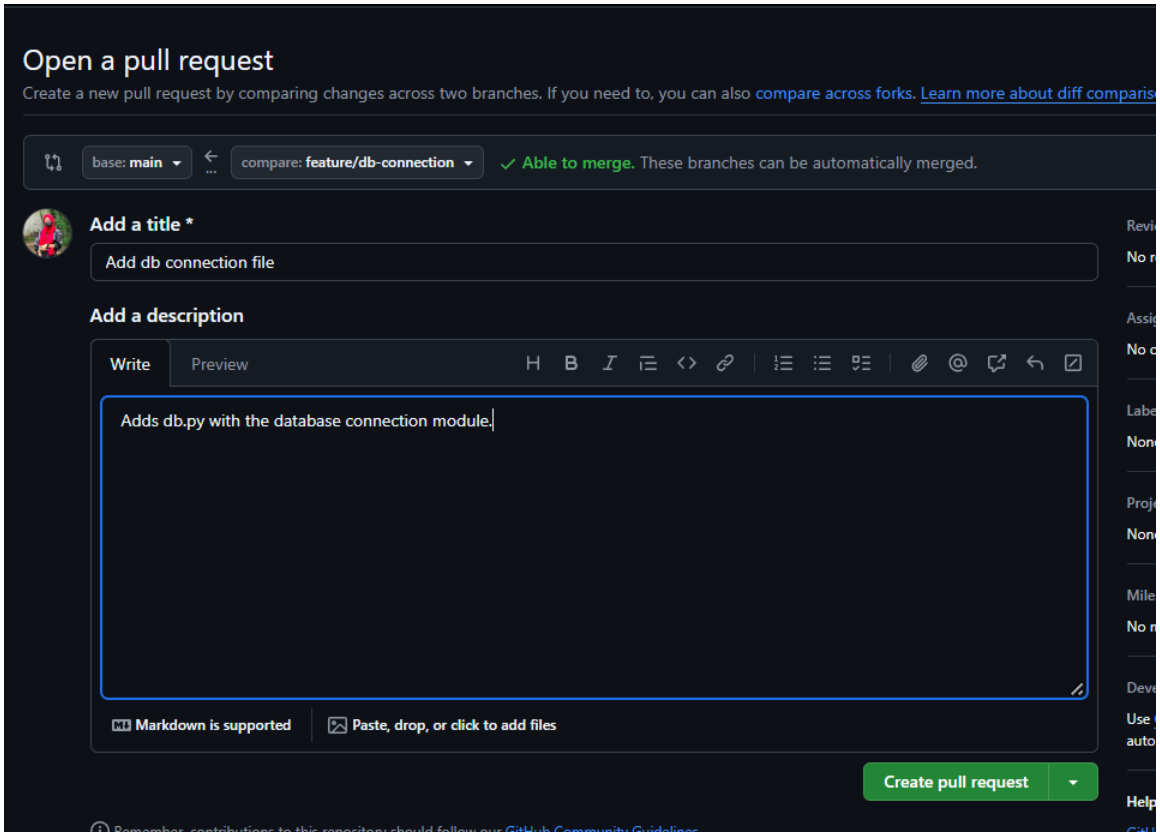


Figure 43: Pull request form, with the title entered

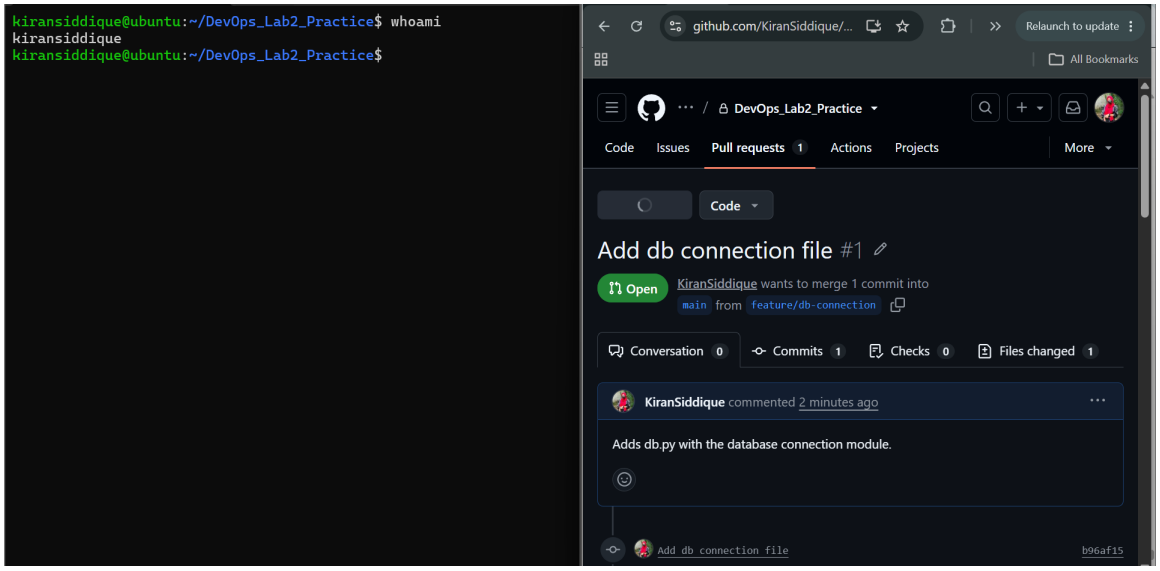


Figure 44: Pull request created

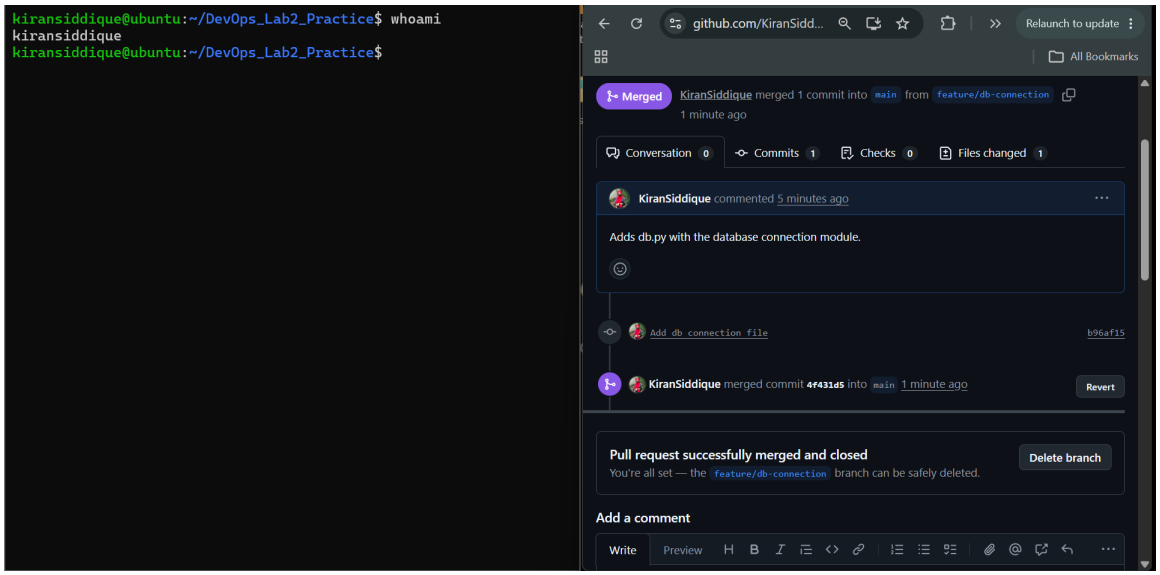


Figure 45: Reviewing and merging the Pull Request

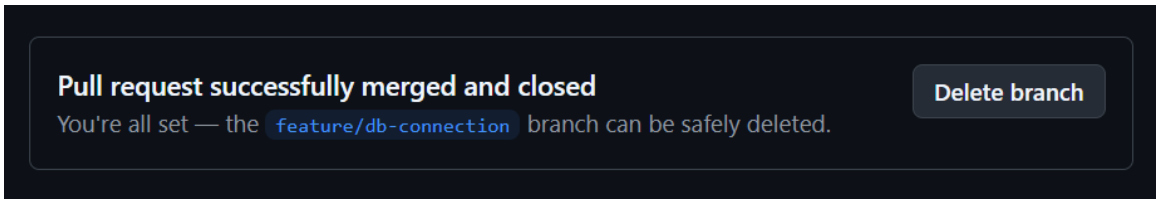


Figure 46: Pull request successfully merged and closed

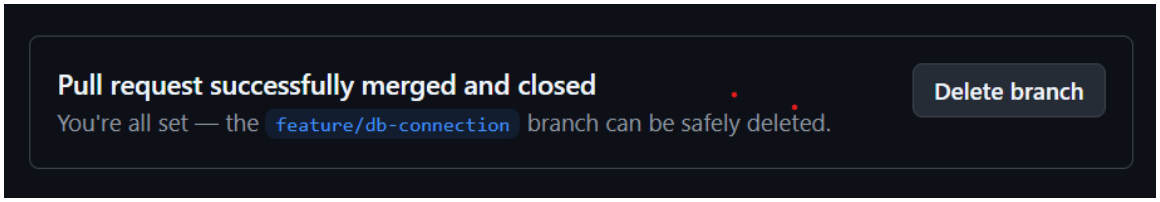


Figure 47: Delete branch button shown after the merge

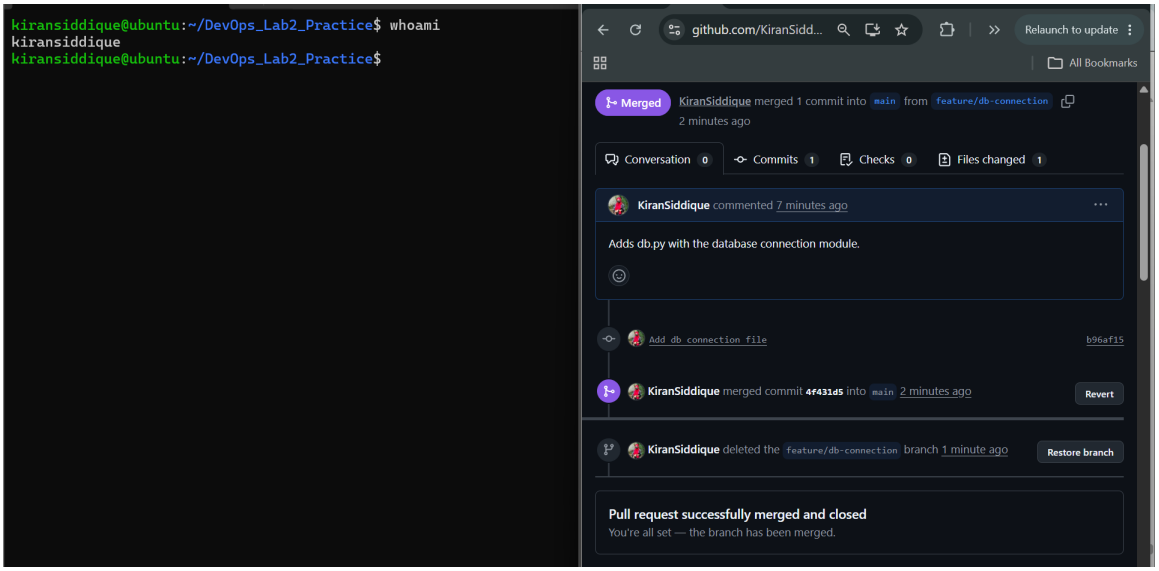


Figure 48: Merged branch deleted from the GitHub GUI

13. Task 11: Branch Strategy (develop and staging)

A standard branching model was built: feature and bugfix branches are merged into develop, develop is merged into staging for testing, and staging is merged into main for production. Feature work was first done on a feature/* branch, as shown below.

- Switched to the existing feature/login branch, edited login.py, staged it, and committed it with the message Add login feature.
- Created develop and staging, and checked the branch list.
- Merged completed feature work into develop.
- Merged develop into staging for testing.
- Merged staging into main as the final production merge.

```
git checkout feature/login
git add login.py
git commit -m "Add login feature"
git checkout -b develop
git push origin develop
git checkout -b staging
git push origin staging
git checkout staging
git merge develop
git push origin staging
git checkout main
git merge staging
git push origin main
```

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout featur
e/login
Switched to branch 'feature/login'
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ echo "login feature
" > login.py
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git add login.py
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Add
login feature"
[feature/login 3784e0c] Add login feature
 1 file changed, 1 insertion(+)
 create mode 100644 login.py
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout bugfix
/typo
Switched to branch 'bugfix/typo'
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ echo "typo fixed" >
 fix.txt
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git add fix.txt
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Fix
typo"
[bugfix/typo 2827763] Fix typo
 1 file changed, 1 insertion(+)
 create mode 100644 fix.txt
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 49: Working on feature/login: editing login.py, staging and committing

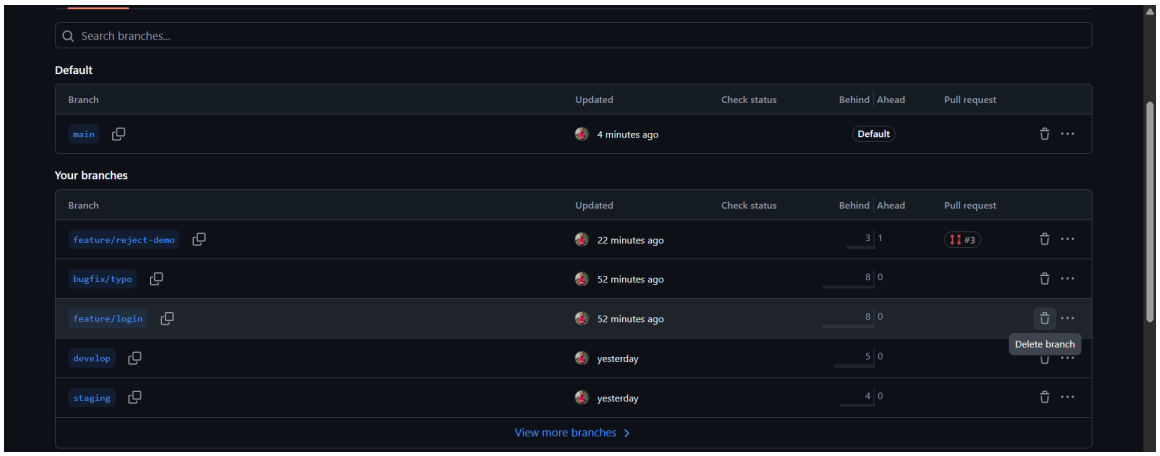


Figure 50: GitHub branch page listing the feature and bugfix branches

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git branch develop
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git branch staging
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git branch feature/
login
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git branch bugfix/t
ypo
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git push origin dev
elop staging feature/login bugfix/typo
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:KiranSiddique/DevOps_Lab2_Practice.git
 * [new branch]      develop -> develop
 * [new branch]      staging -> staging
 * [new branch]      feature/login -> feature/login
 * [new branch]      bugfix/typo -> bugfix/typo
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git branch -a
  bugfix/typo
  bugfix/user-auth-error
  develop
  feature-1
  feature/db-connection
  feature/login
* main
  staging
  remotes/origin/HEAD -> origin/main
  remotes/origin/bugfix/typo
  remotes/origin/bugfix/user-auth-error
  remotes/origin/develop
  remotes/origin/feature-1
  remotes/origin/feature/login
  remotes/origin/main
  remotes/origin/staging
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 51: Branch list showing the workflow branches

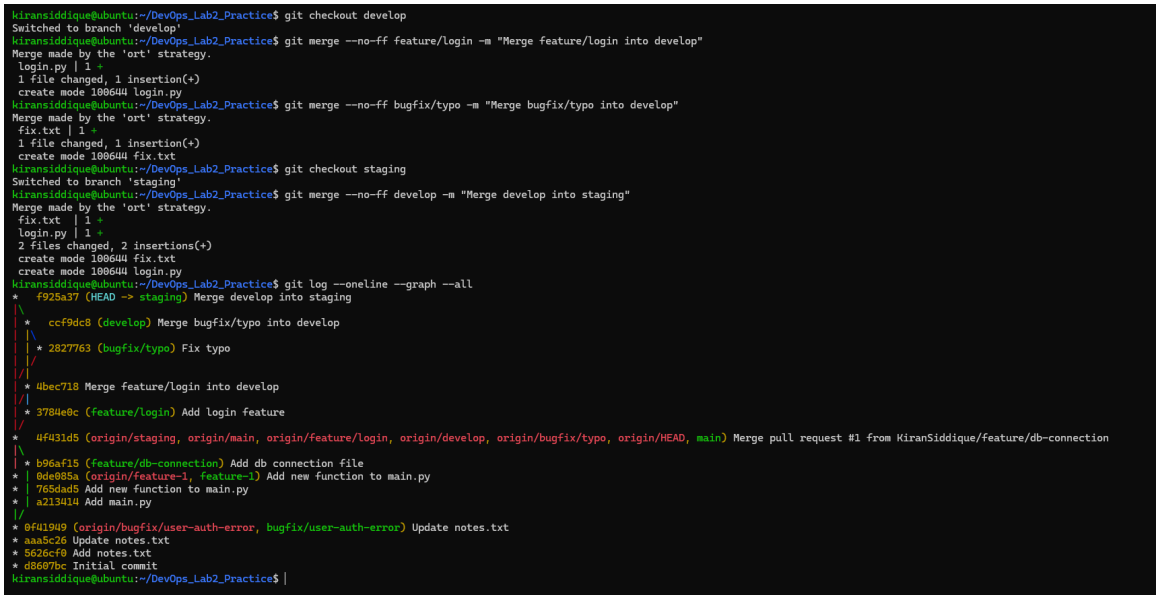


Figure 52: Merges into develop and staging

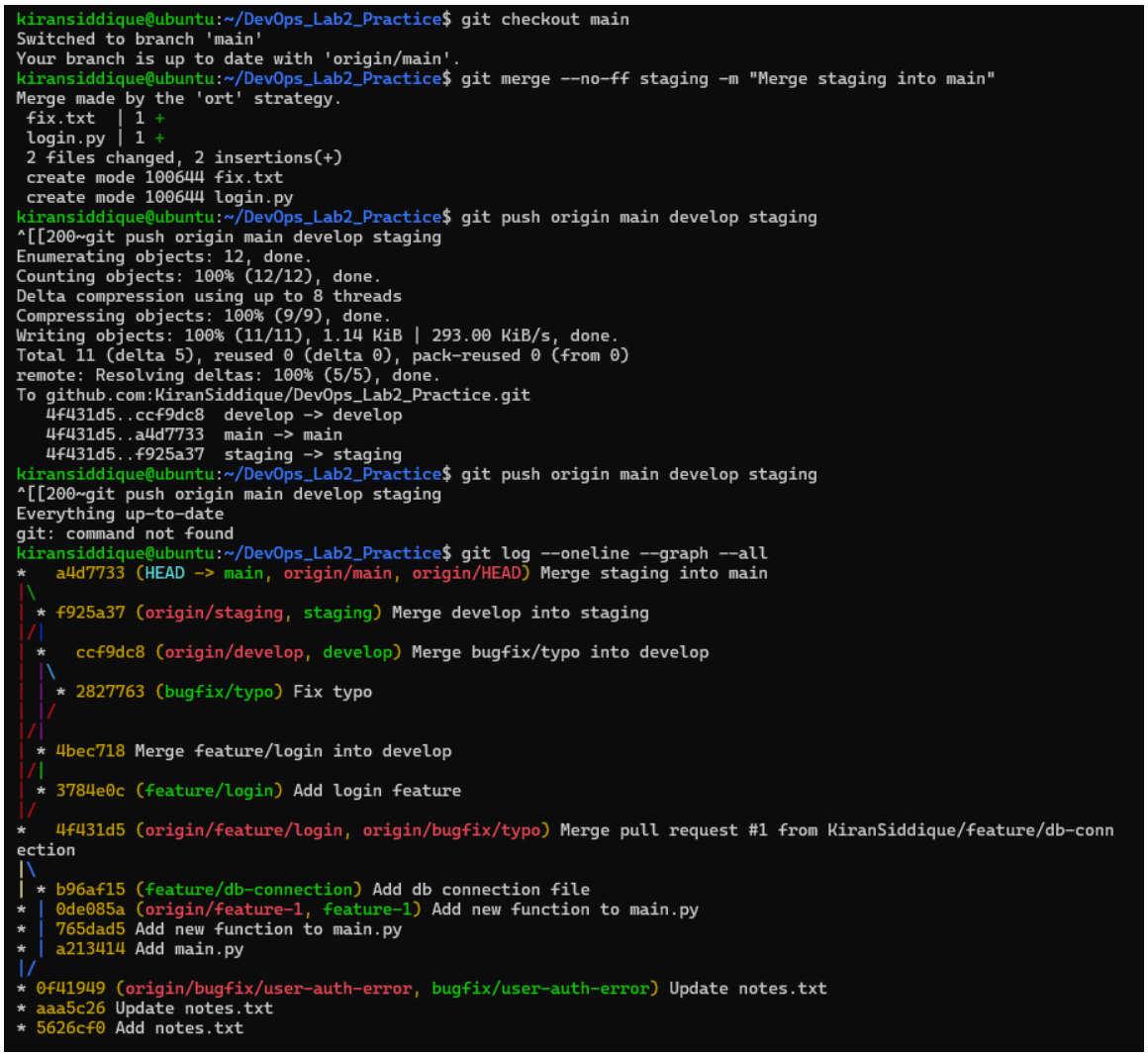


Figure 53: Final merge into main

14. Task 12: Code Review Workflow

A second GitHub account (kiransiddique406-source) was added as a collaborator and used as the reviewer. The lab asks for each review state to be shown: changes requested, a rejected practice PR, author updates, and approval followed by a merge.

14.1 Inviting the Reviewer as a Collaborator

- On the repository, opened Settings > Collaborators (under Access) and selected Add people.
- Entered the second account's username kiransiddique406-source and sent the invitation.
- The second account received the invitation, opened it, and accepted it.
- After acceptance, GitHub confirmed that the account now has push access to the repository.

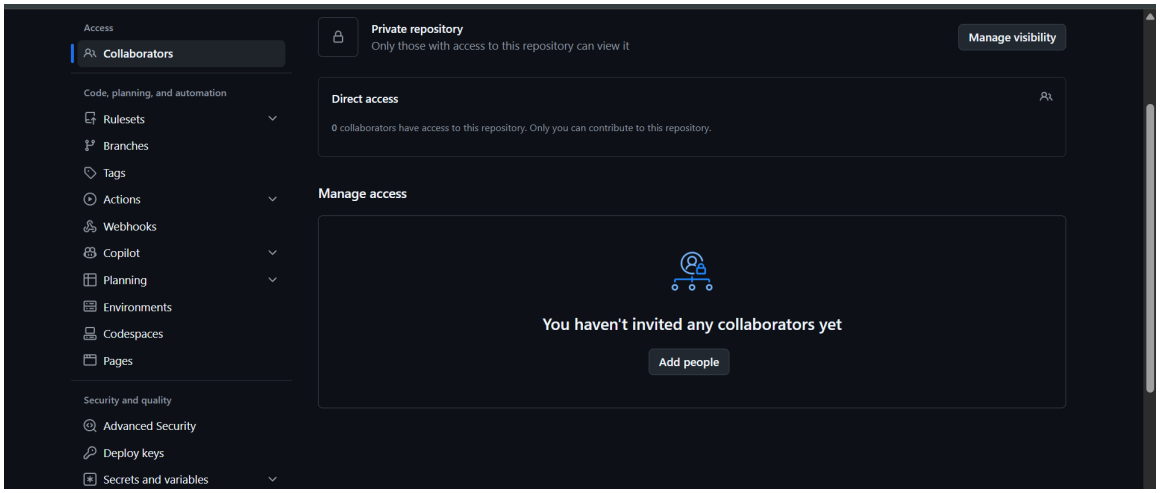


Figure 54: Access page with Manage visibility and the Collaborators section

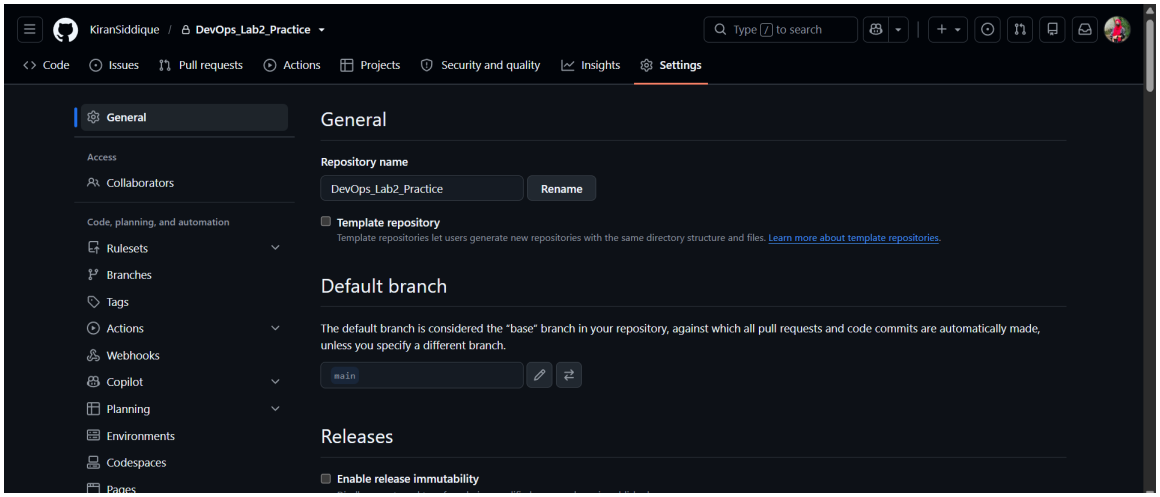


Figure 55: Settings page: General and Access options for managing collaborators

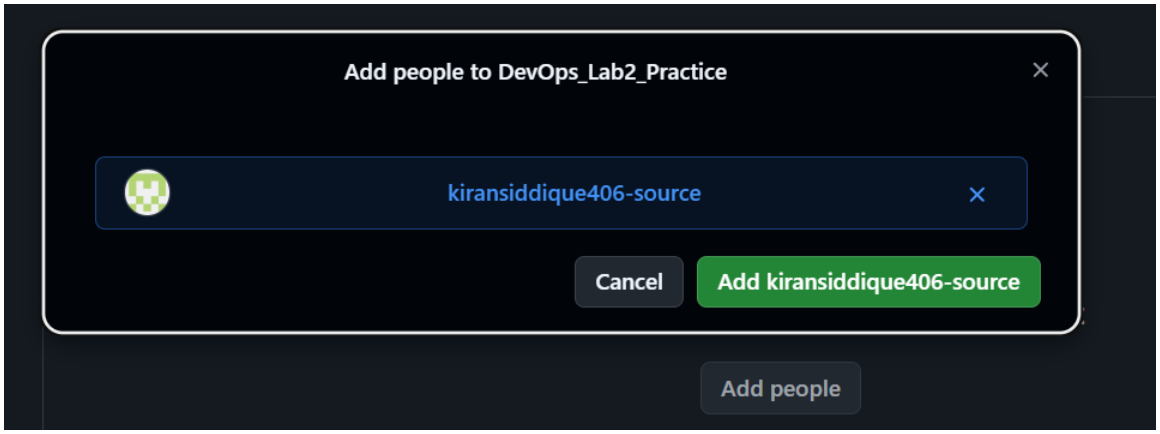


Figure 56: Add people dialog with the second account username

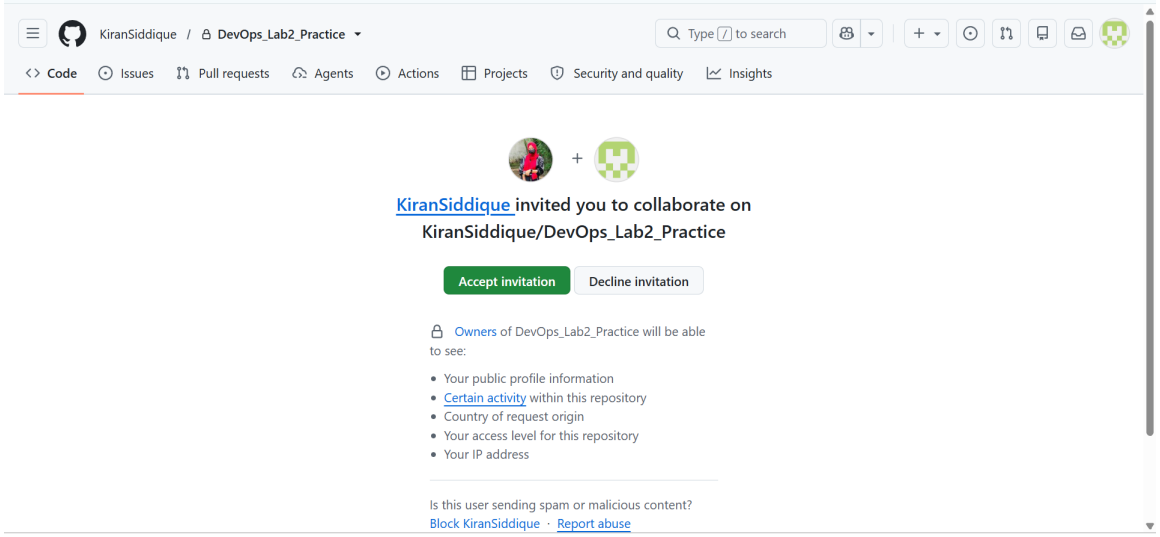


Figure 57: Invitation to collaborate on KiranSiddique/DevOps_Lab2_Practice

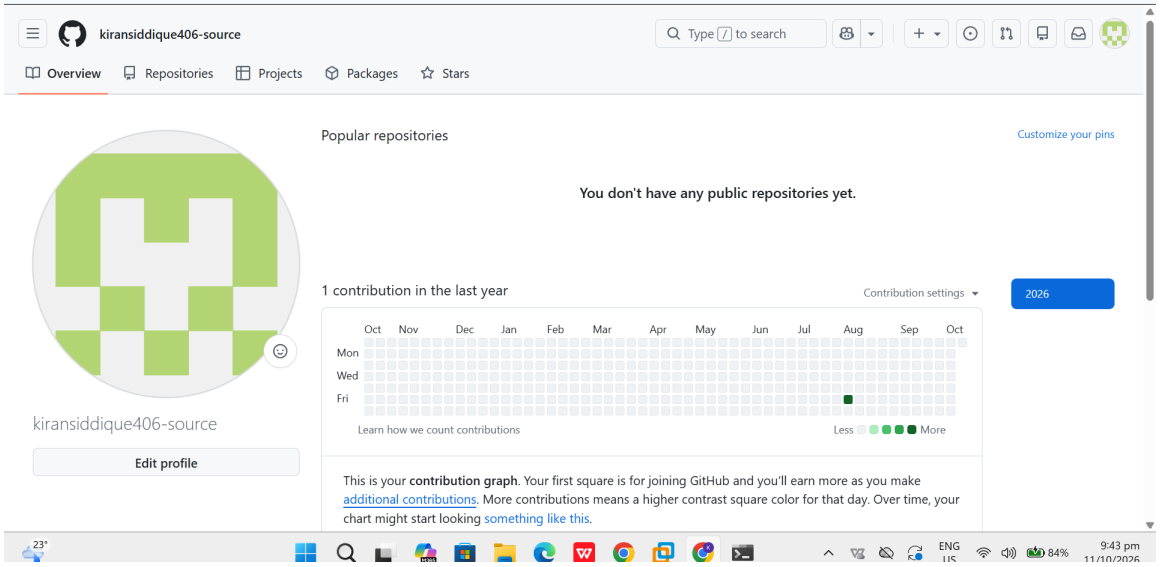


Figure 58: Second account's GitHub profile

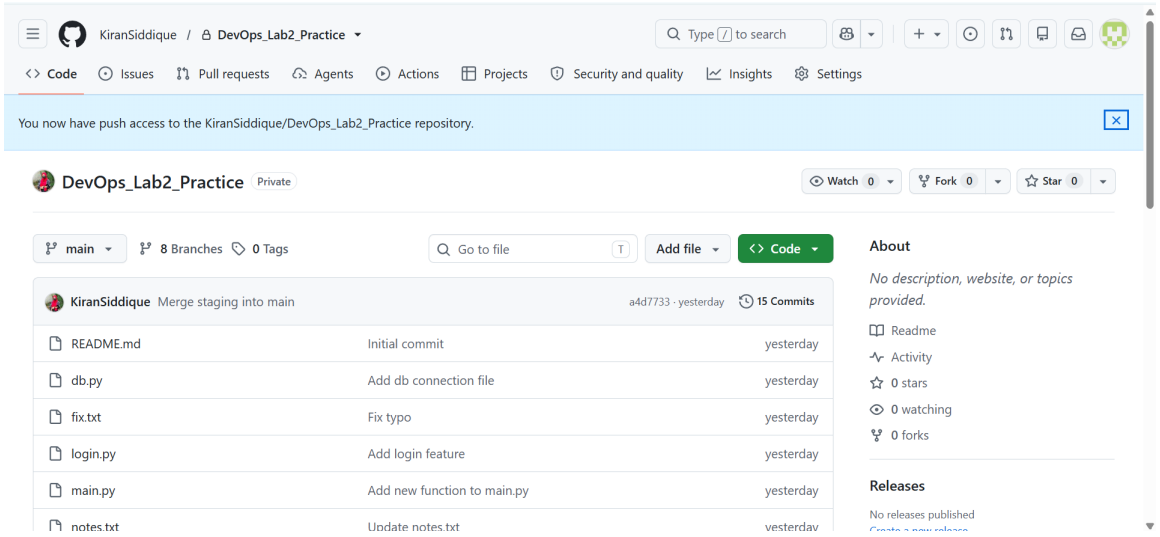


Figure 59: Collaborator accepted the invitation and now has push access

14.2 Creating the Pull Request and Assigning a Reviewer

- The author account used the branch feature/review-demo, which contains review.txt (+18 lines).
- Opened a Pull Request from feature/review-demo into main with the title Add review demo file and a description: Adds review.txt to demonstrate the code review workflow.
- Assigned kiransiddique406-source as a reviewer. The Reviewers panel supports up to 15 reviewers.
- The reviewer account received the request (KiranSiddique requested your review).

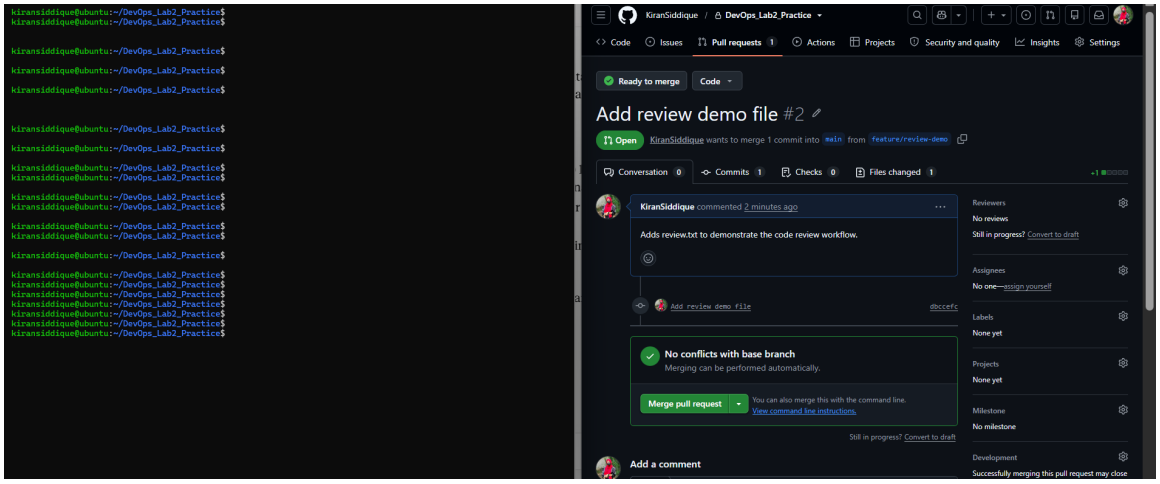


Figure 60: Pull Request title and description

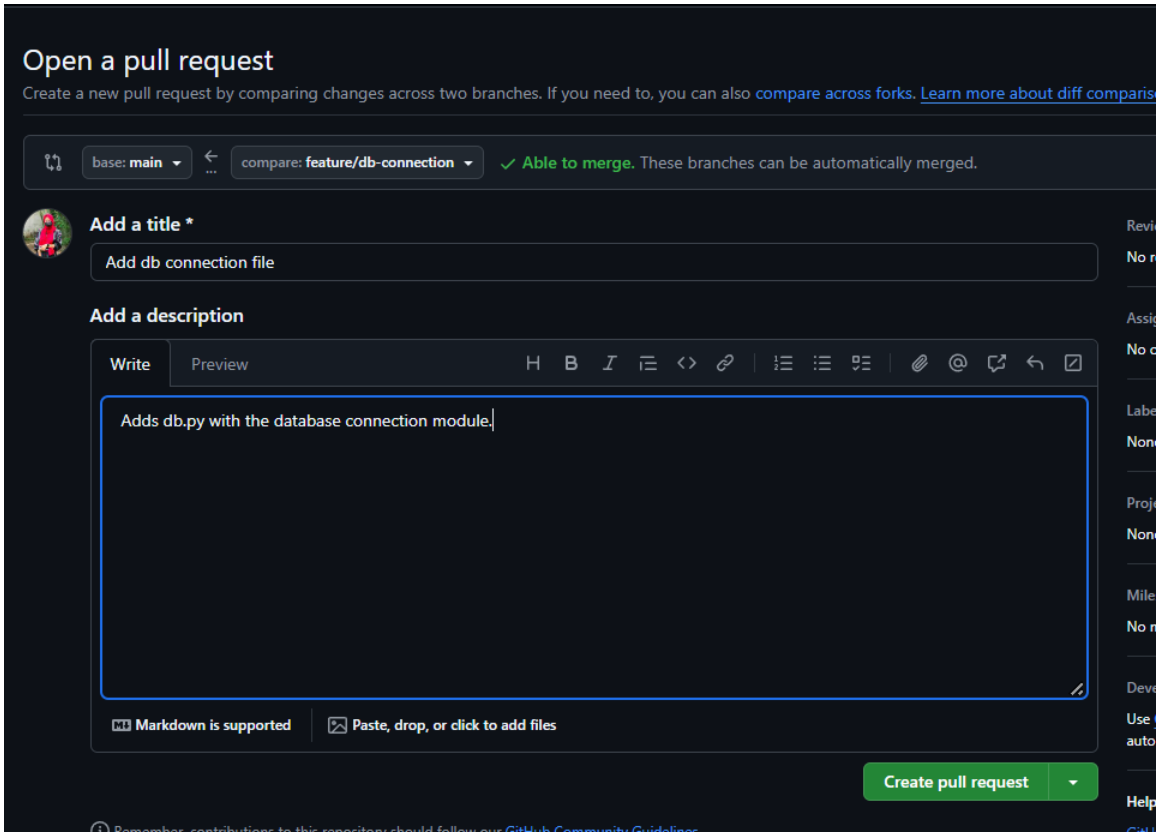


Figure 61: Pull request comparing feature/review-demo with main

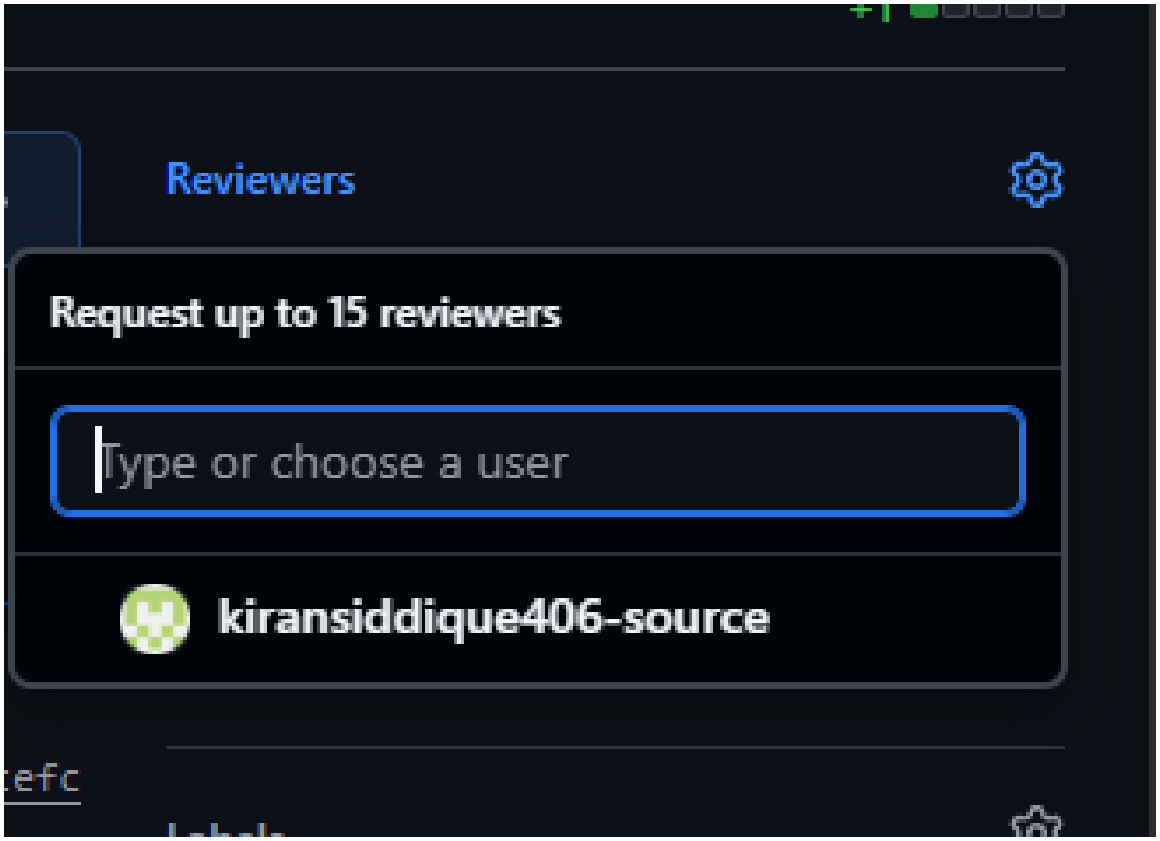


Figure 62: Reviewers panel with the second account selected

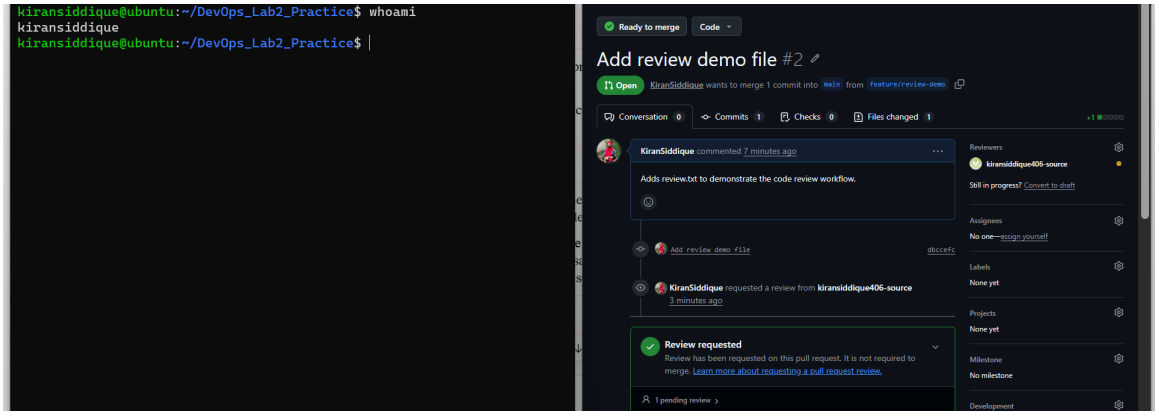


Figure 63: Reviewer assigned to the Pull Request

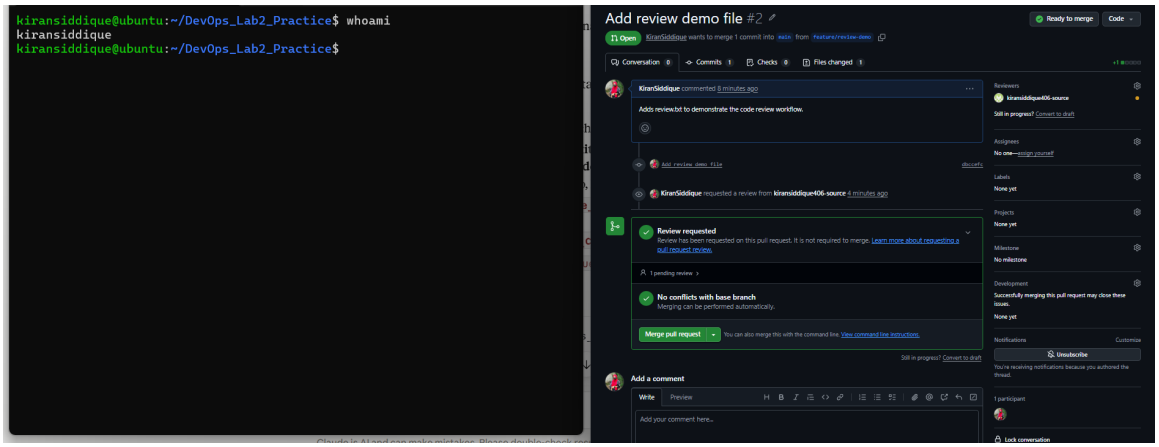


Figure 64: Pull Request #2 opened by KiranSiddique, with the commit from feature/review-demo

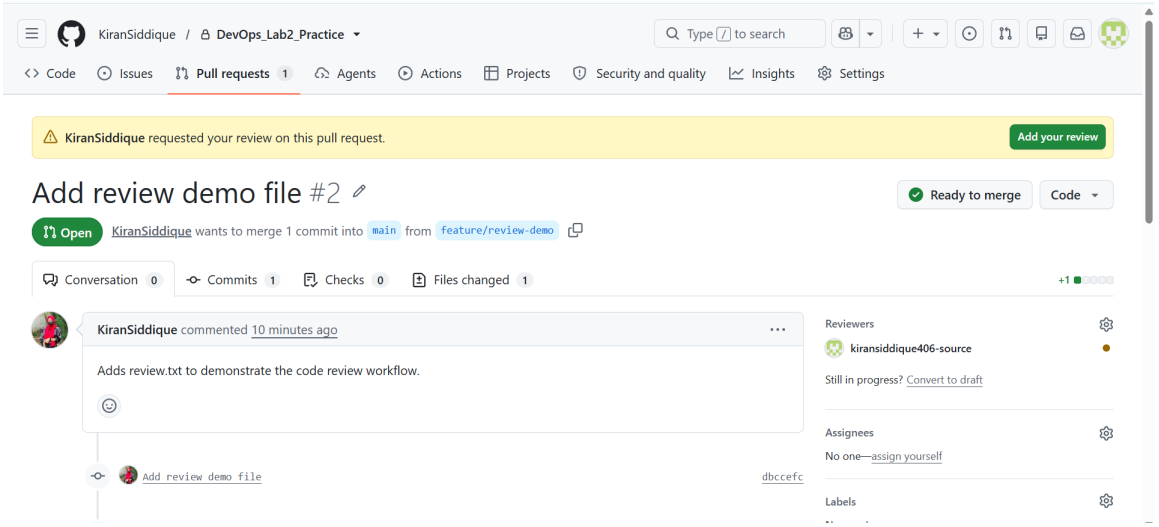


Figure 65: Reviewer's notification: review requested on the Pull Request

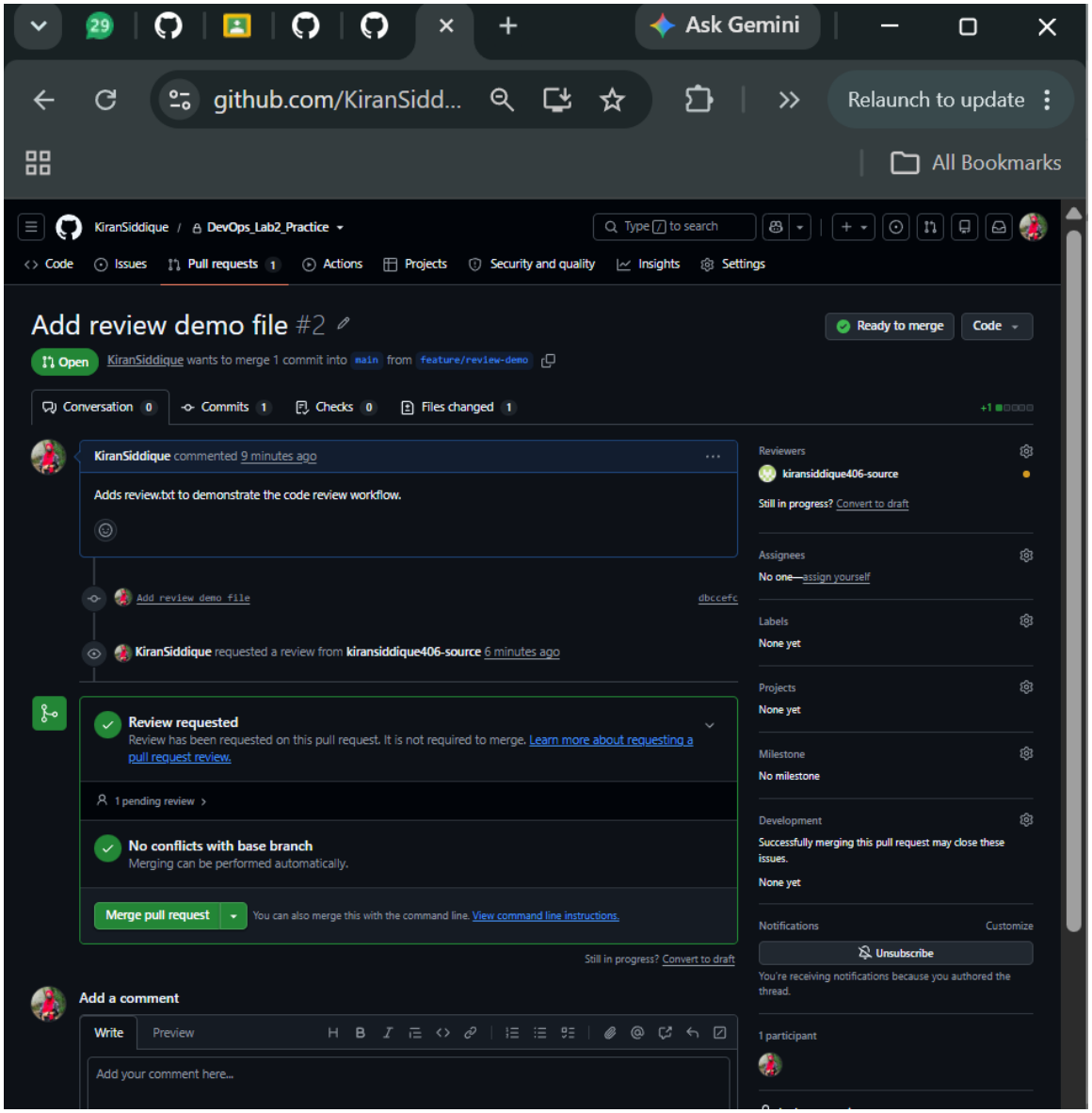


Figure 66: Pull Request #2 with the review requested and the merge button

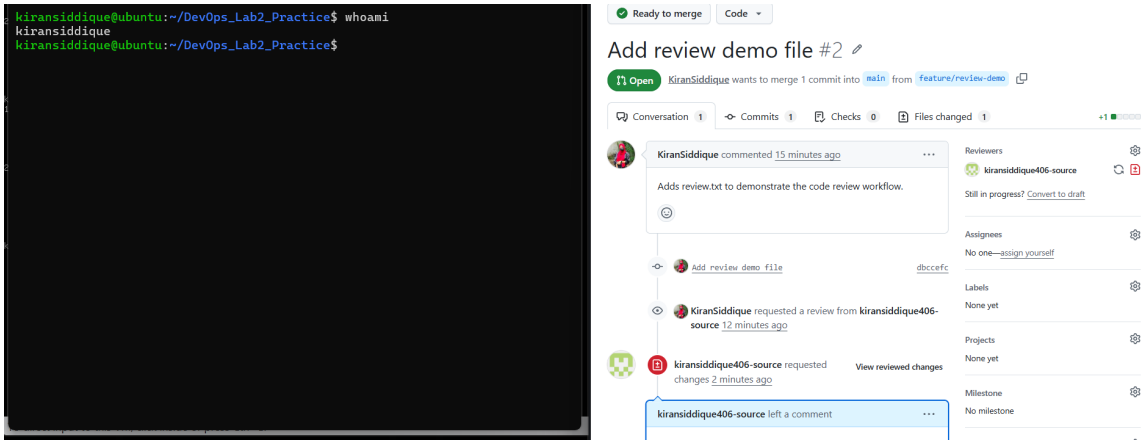


Figure 67: Terminal (whoami) and the Pull Request #2 page, showing the request to merge into main

14.3 Reviewer Actions: Request Changes and Reject

- The reviewer opened Files changed, viewed review.txt (+18 lines), and marked the file as viewed.
- Selected Finish your review and chose Request changes, with the comment: Changes requested, please see my comment.
- The Pull Request shows the state: kiransiddique406-source requested changes, with the comment Changes requested, please see my comment.
- A separate practice branch, feature/reject-demo, was created so that rejection could be shown without touching the main work. That practice Pull Request was closed.

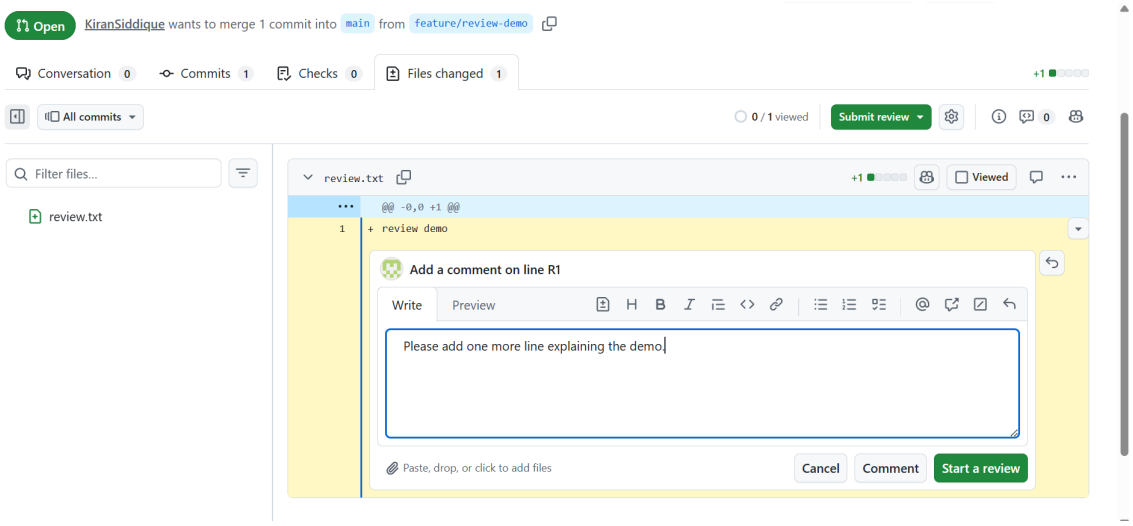


Figure 68: Reviewer opened Files changed and reviewed review.txt

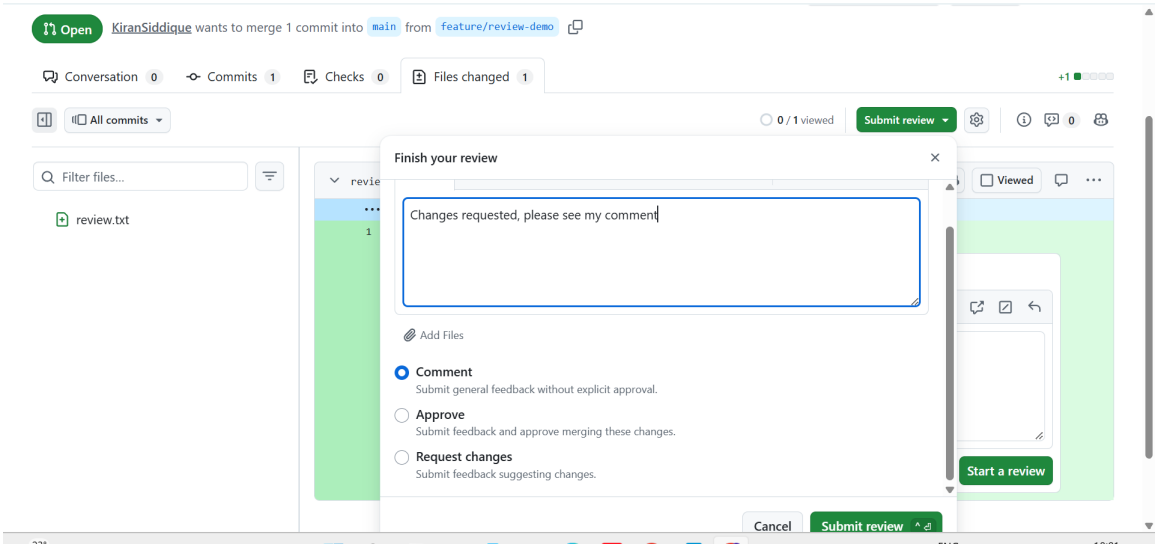


Figure 69: Finish your review dialog

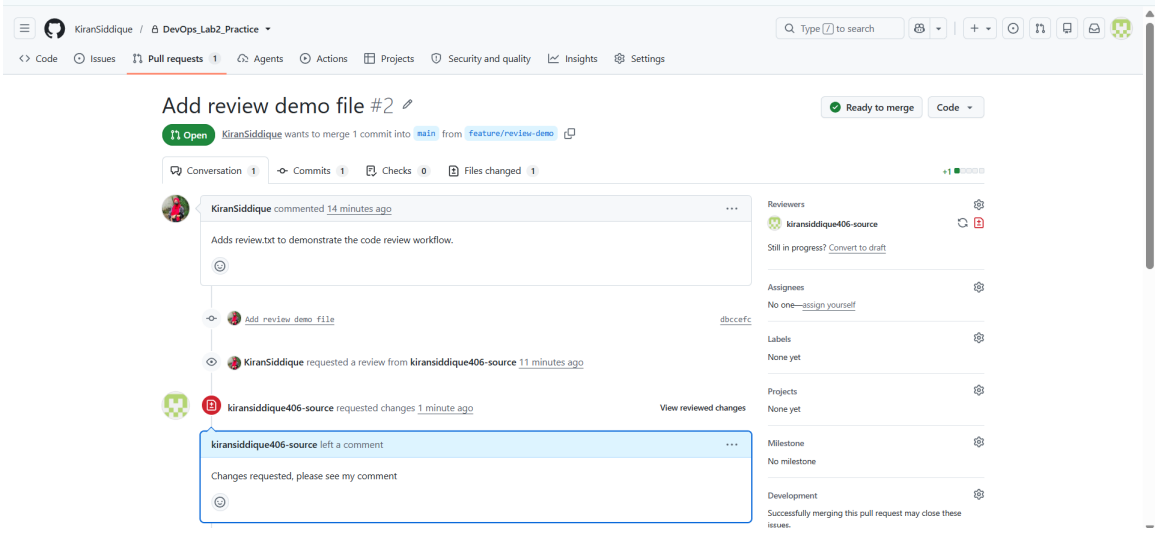


Figure 70: Pull Request after the reviewer requested changes

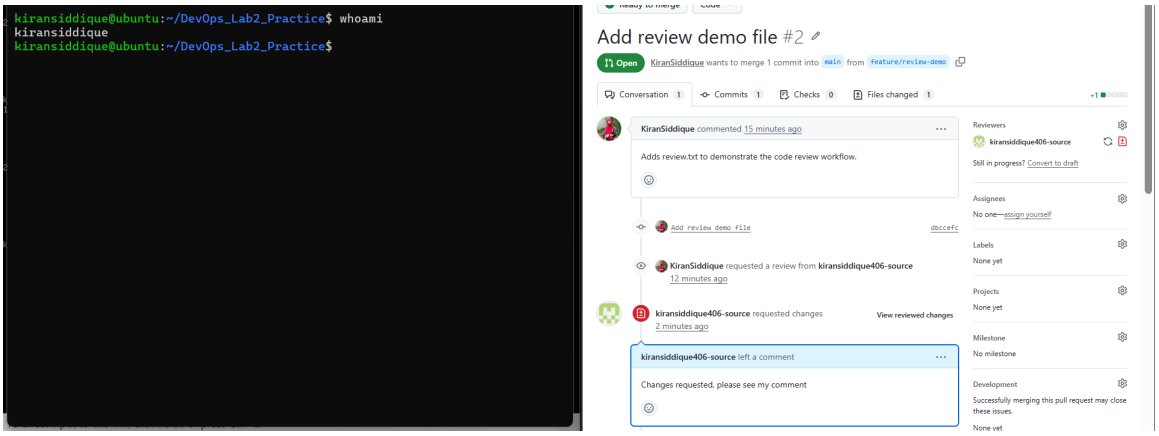


Figure 71: Reviewer requested changes with a comment

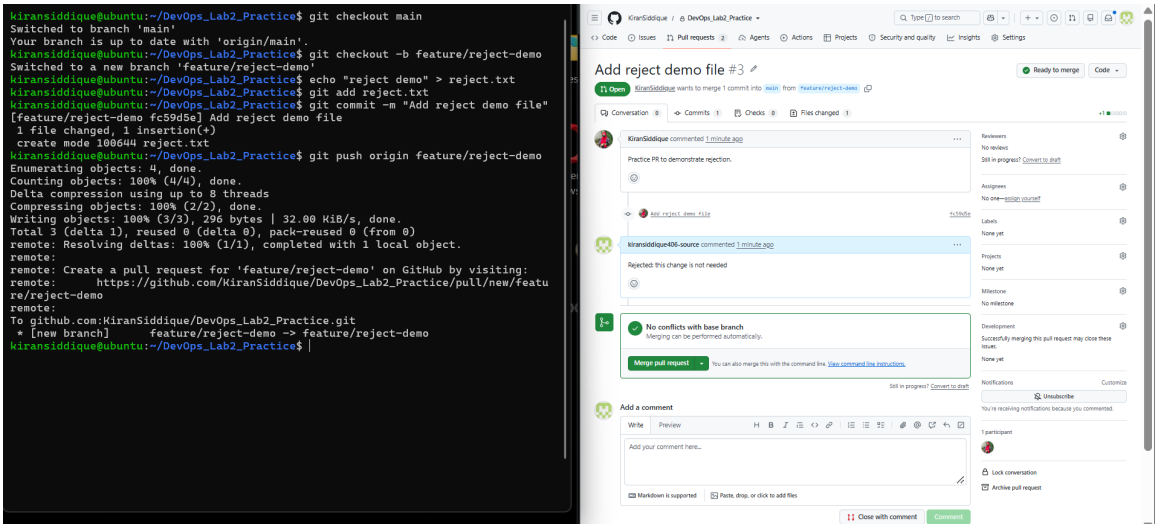


Figure 72: Author terminal: creating the practice branch feature/reject-demo



Figure 73: Terminal window of the author's Ubuntu VM

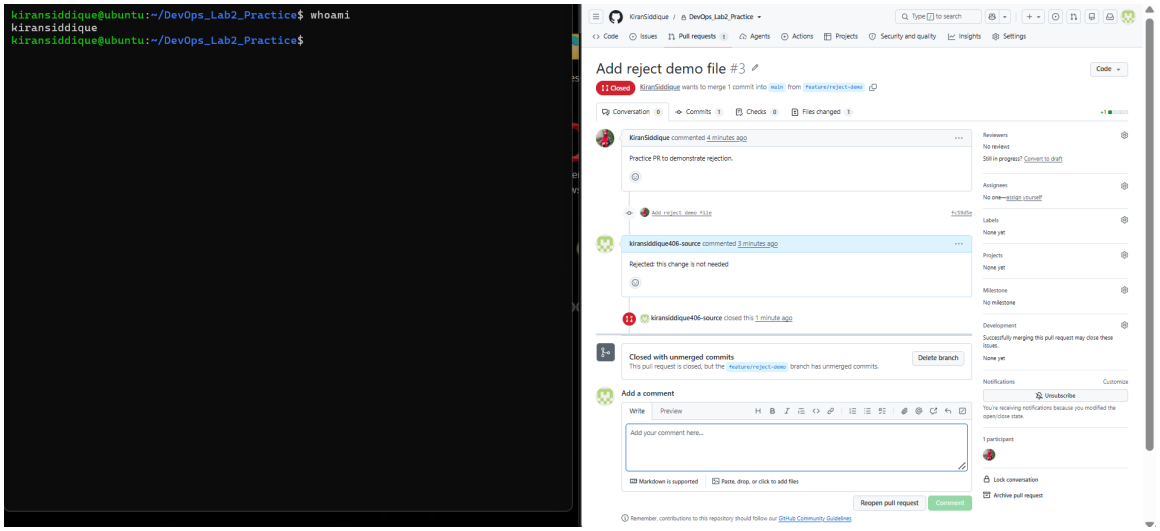


Figure 74: Practice Pull Request rejected and closed

14.4 Author Actions: Fix Issues and Push New Commits

- The author switched to feature/review-demo, fixed the review comments, committed the changes, and pushed.
- The Pull Request updated automatically with the new commit, so it now shows 2 commits.
- The reviewer replied on the Pull Request: Looks good now.

```
git checkout feature/review-demo
git add .
git commit -m "Address review comments"
git push origin feature/review-demo
```

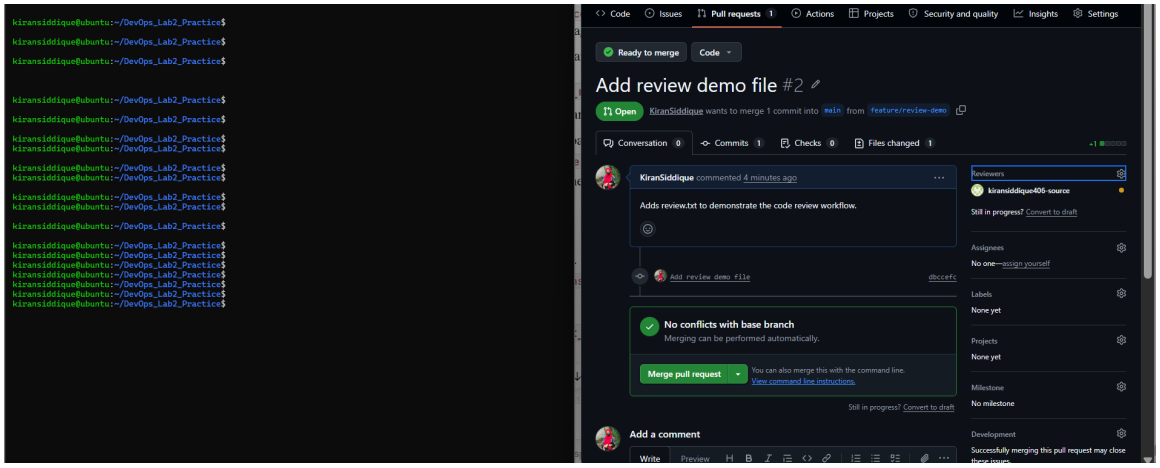


Figure 75: Repository Pull requests tab with the terminal open

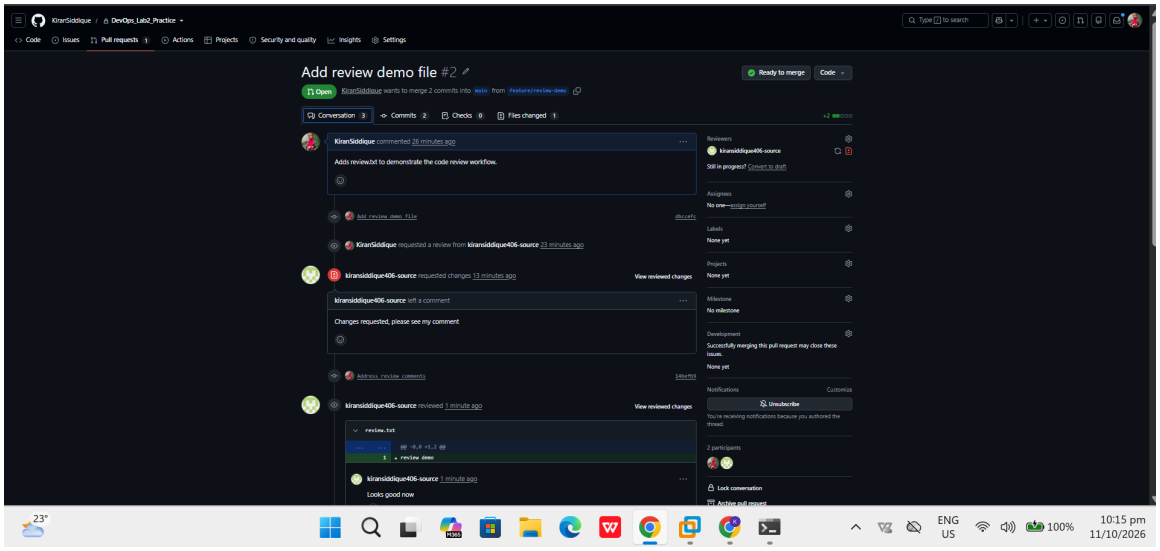


Figure 76: Pull Request now merging 2 commits into main

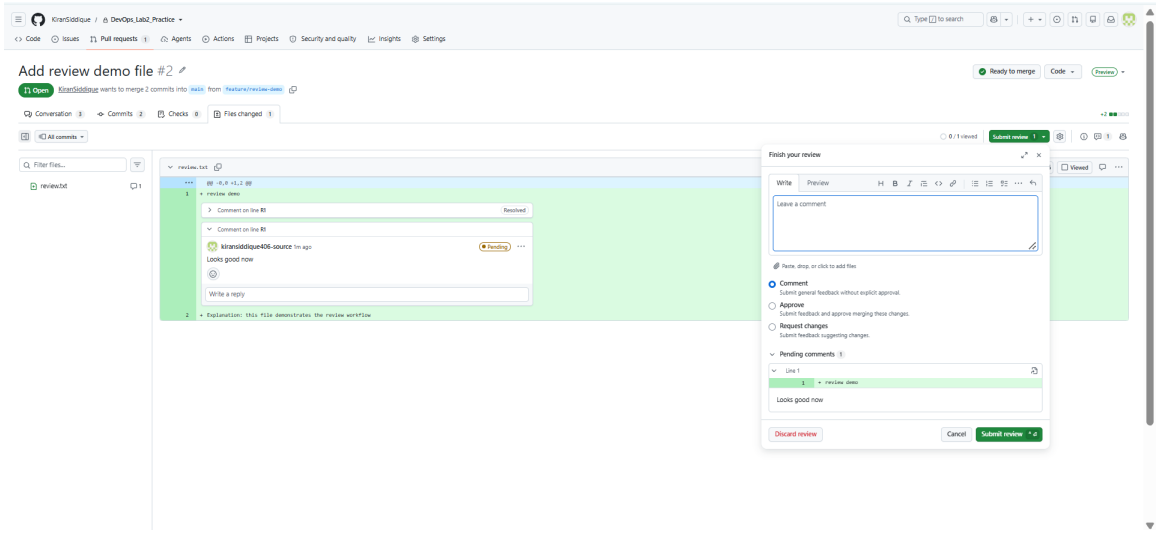


Figure 77: Pull Request with 2 commits (author account)

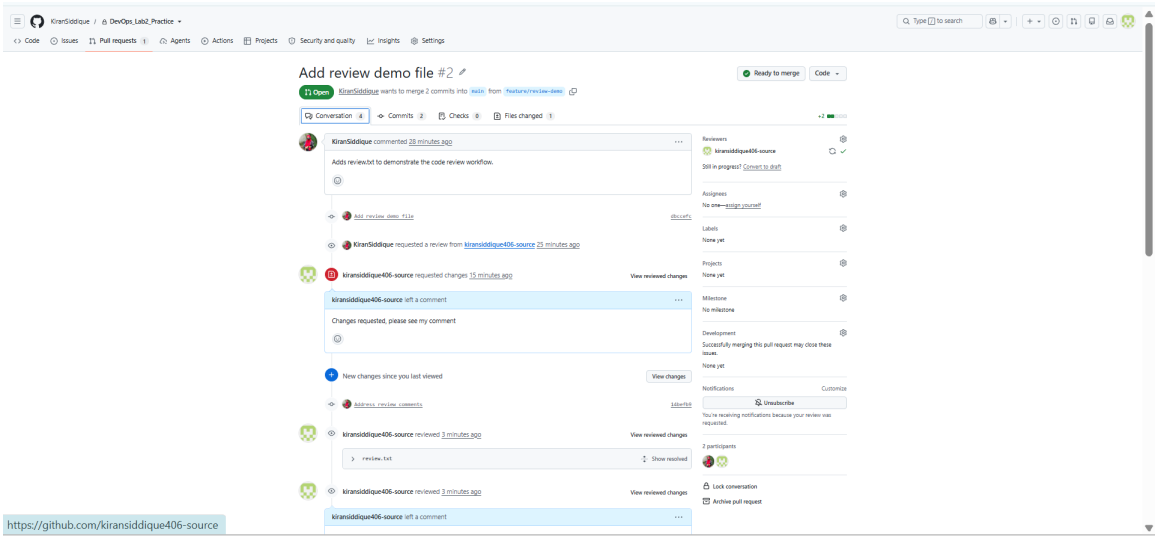


Figure 78: Pull Request with 2 commits after the update

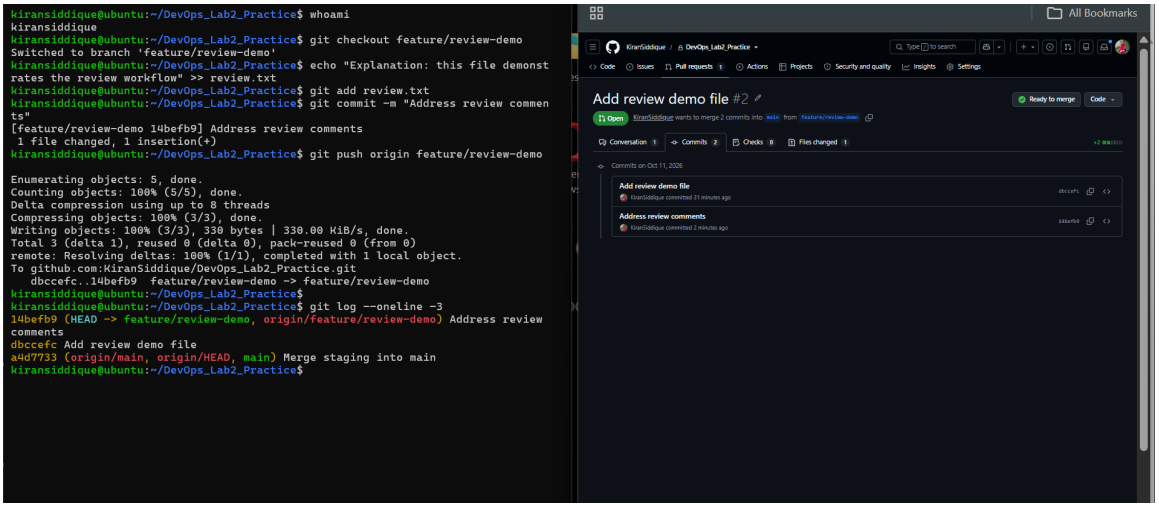


Figure 79: Pull Request updated with new commits

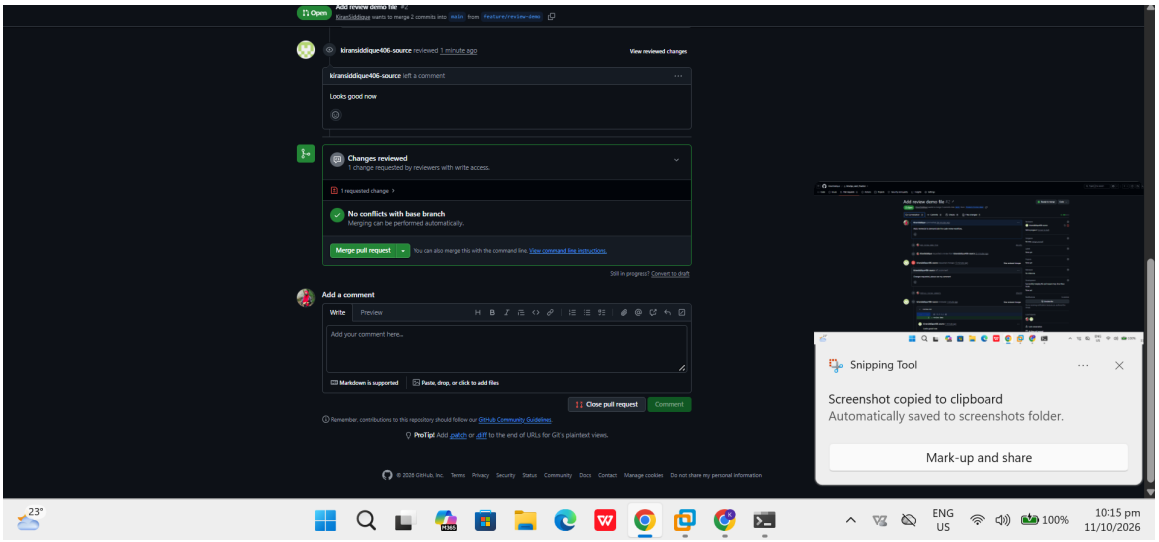


Figure 80: Reviewer comment: Looks good now

14.5 Approval and Merge

- The reviewer approved the Pull Request. The merge status shows 1 approving review by reviewers with write access.
- Checked the merge locally with git checkout feature/review-demo and the merge preview on GitHub.
- Chose the merge method. The options are Create a merge commit, Squash and merge, and Rebase and merge; a merge commit was used.
- Confirmed the merge, and the Pull Request changed to Merged.

```
git checkout feature/review-demo
whoami
```

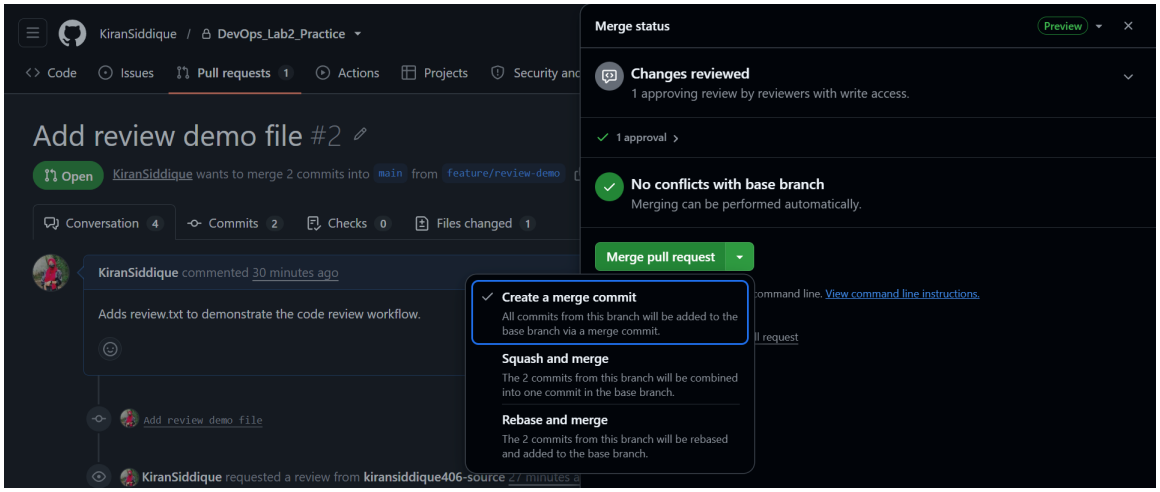


Figure 81: Merge status: 1 approving review, no conflicts

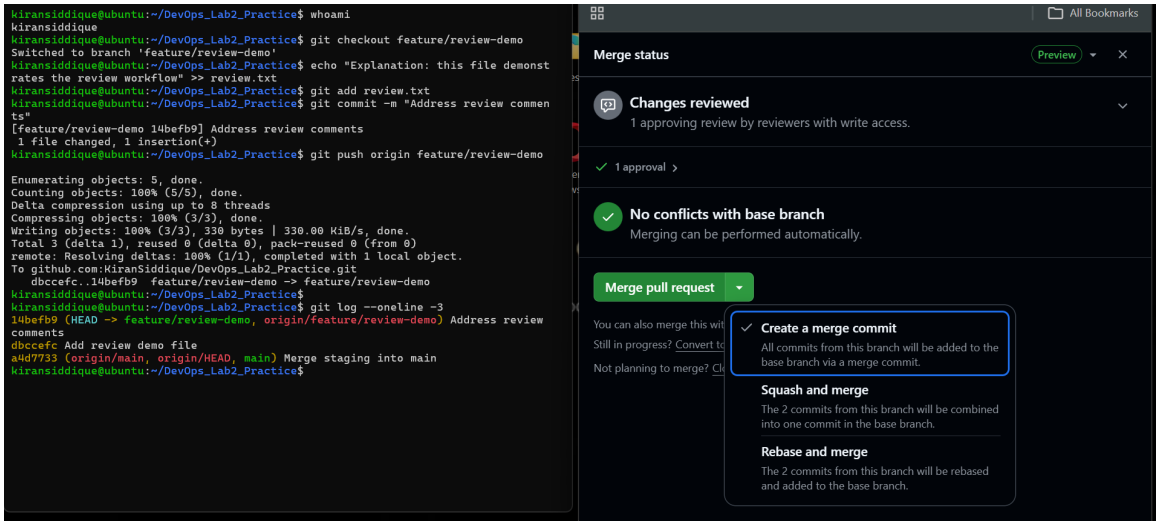


Figure 82: Local checkout of feature/review-demo before the merge

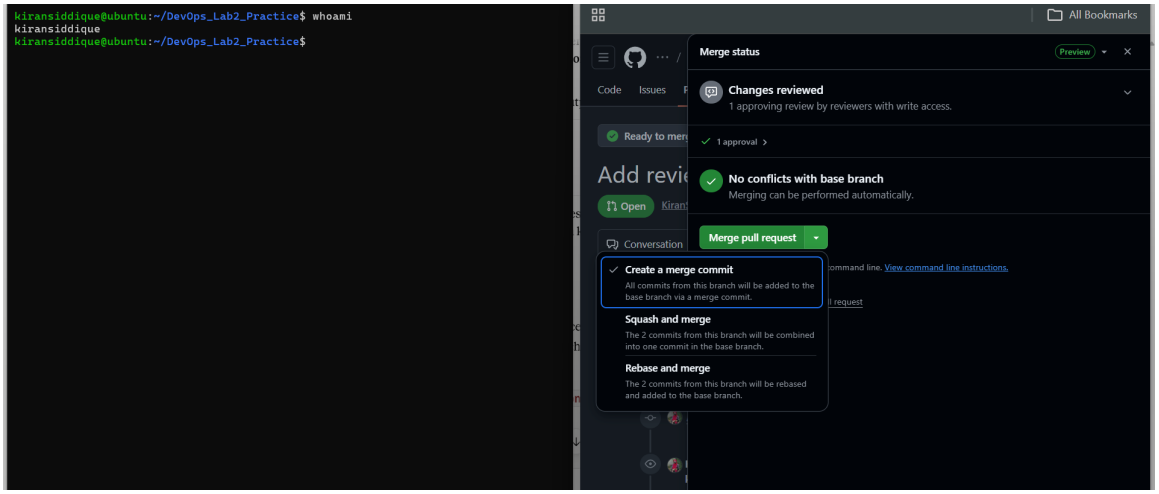


Figure 83: Merge method options on GitHub

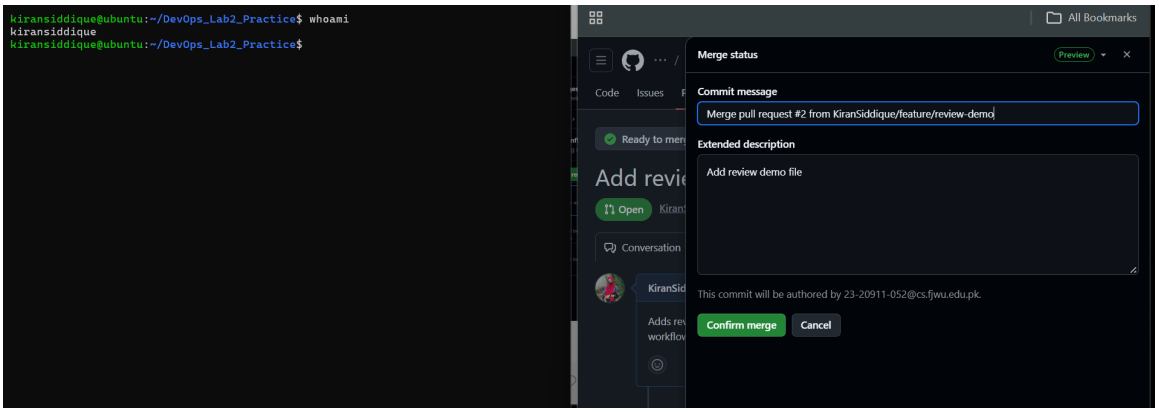


Figure 84: Merge confirmation

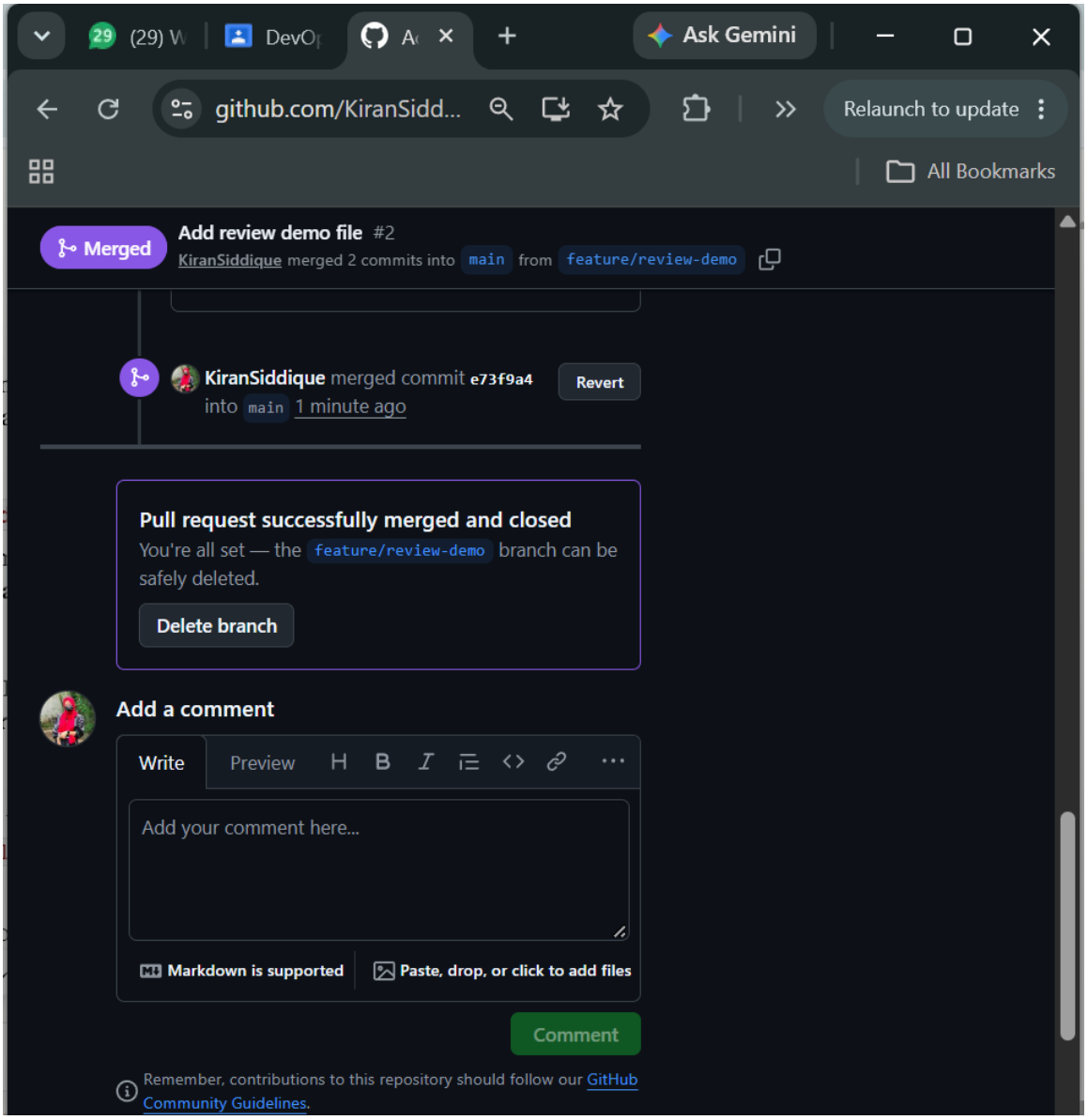


Figure 85: Pull Request #2 marked as Merged

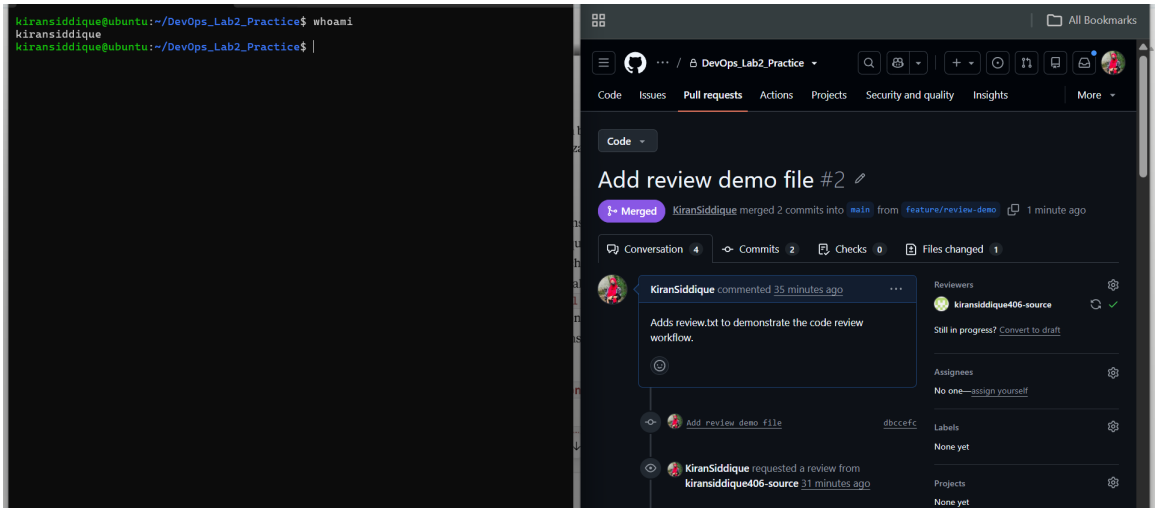


Figure 86: Pull Request merged

14.6 Branch Handling After Merge

The merged feature/review-demo branch was deleted with the Delete branch button on the Pull Request page.

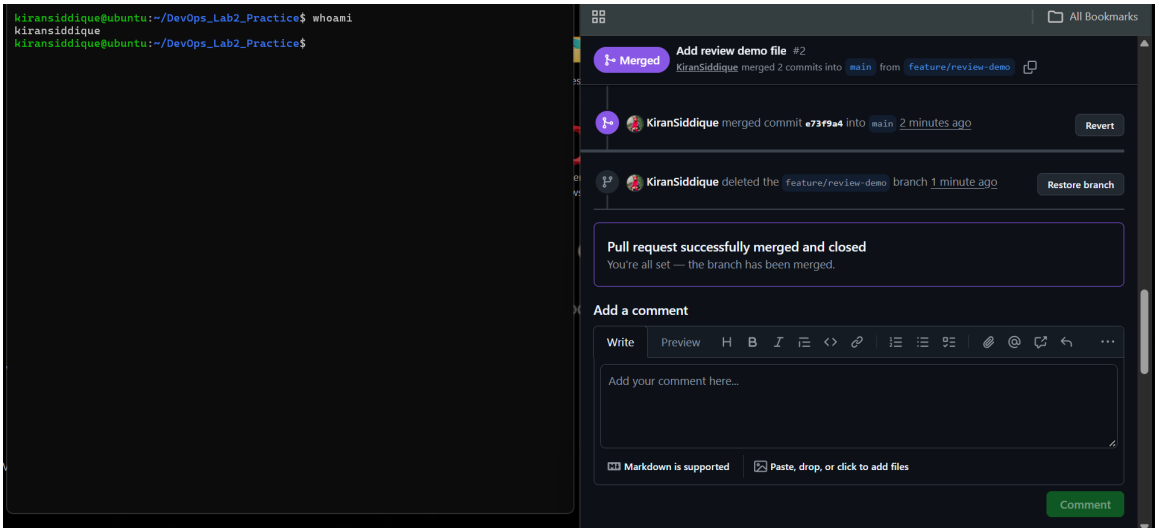


Figure 87: Merged PR branch deleted

15. Task 13: Branch Cleanup Best Practices

Merged branches were removed from GitHub and from the local repository. Deleting merged branches keeps the branch list short and avoids confusion about which work is still active.

15.1 Remote Branch Deleted from the GUI

The branch was deleted from the GitHub Branches page using the delete button next to the branch name.

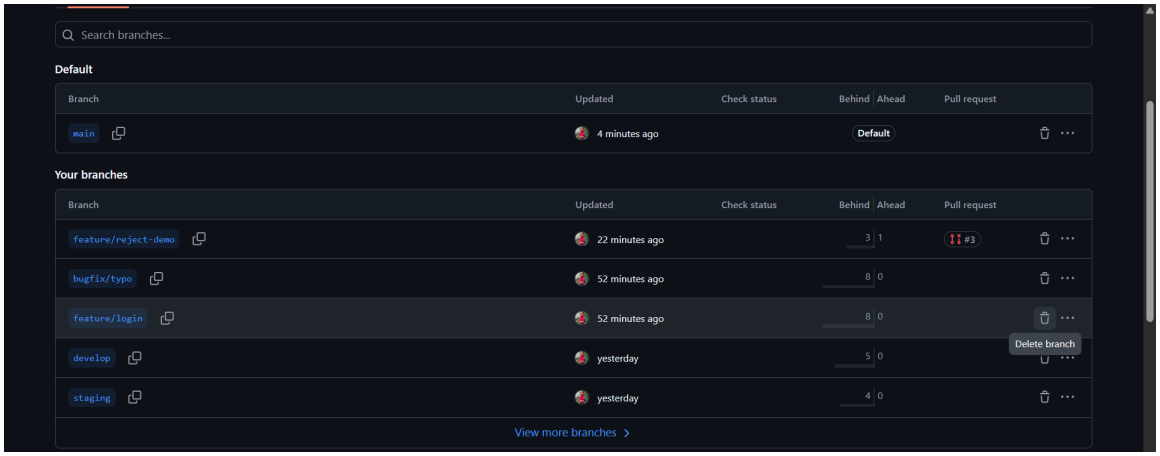


Figure 88: GitHub Branches page with the delete option for a merged branch

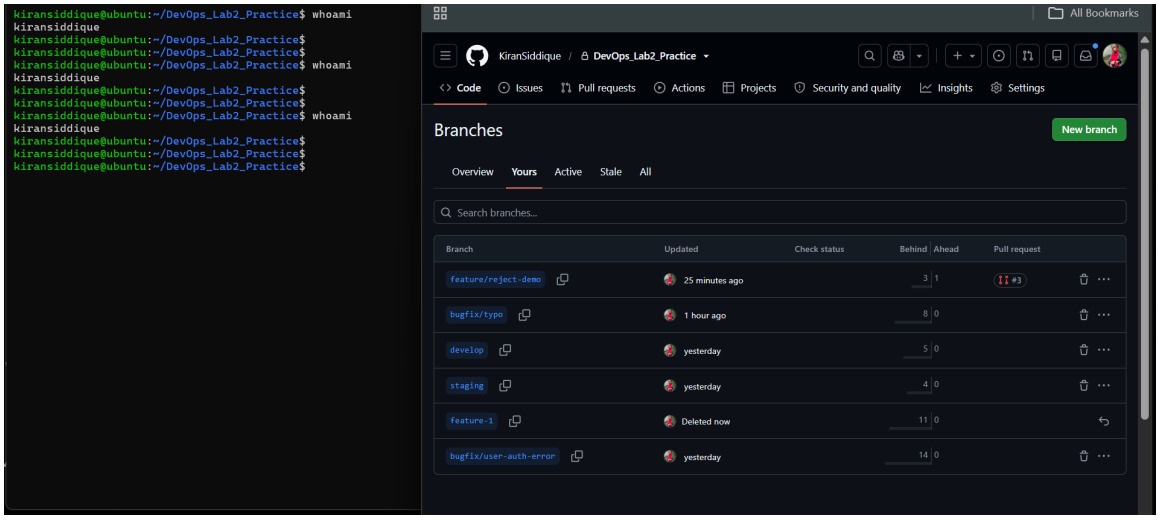


Figure 89: Remote branch deleted from the GitHub UI

15.2 Remote Branch Deleted from the Terminal

```
git push origin --delete <branch-name>
```

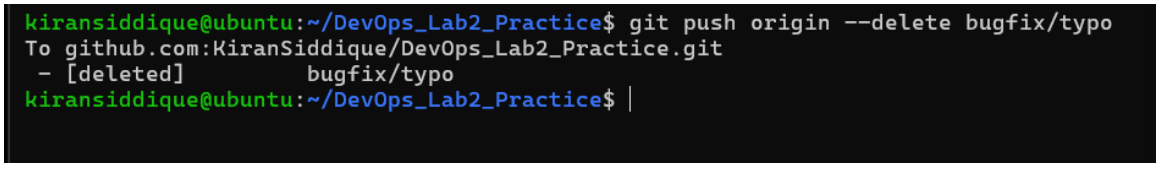


Figure 90: Deleting a remote branch from the terminal

15.3 Updating the Local Repository

```
git checkout main
git pull origin main
git branch -d <branch-name>
git branch -a
```

git branch -d only removes a branch that has already been merged, which protects unmerged work. The branch list afterwards shows the remaining branches.

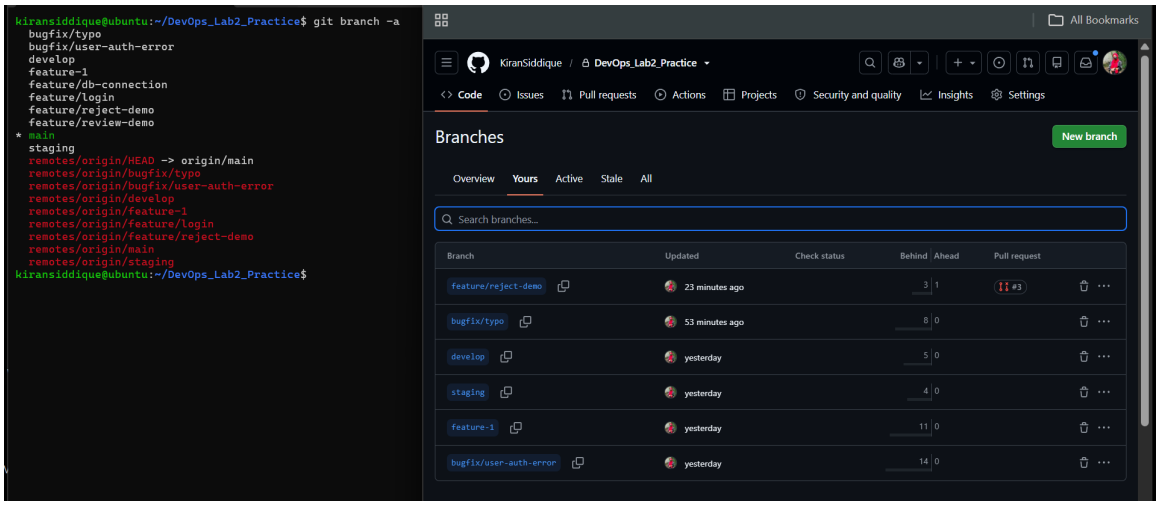


Figure 91: git branch -a after cleanup, showing the remaining local and remote branches

16. Verification and Submission

The final evidence was committed to the DevOps repository under Labs/Lab02 and pushed to the main branch. The following commands were used for the final check.

```
git status
git add Labs/Lab02
git commit -m "Submit Lab 02 evidence"
git push origin main
```

After the push, the Labs/Lab02 folder was checked on GitHub to confirm that the screenshots and files are visible.

17. Bonus: Simulated Team Collaboration

This bonus task simulates two contributors on a shared branch. The main account (KiranSiddique) and the second account (kiransiddique406-source) each added a line to notes.txt on the collab branch. The branch was then merged into main.

17.1 Creating the collab Branch

- Created the collab branch from main and pushed it to origin with upstream tracking.
- Checked the branch list to confirm that collab exists locally and on the remote.

```
git checkout -b collab
git push -u origin collab
git branch -a
```

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout -b collab
Switched to a new branch 'collab'
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git push -u origin collab

Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)

remote:
remote: Create a pull request for 'collab' on GitHub by visiting:
remote:      https://github.com/KiranSiddique/DevOps_Lab2_Practice/pull/new/colla
b
remote:
To github.com:KiranSiddique/DevOps_Lab2_Practice.git
 * [new branch]      collab -> collab
branch 'collab' set up to track 'origin/collab'.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git branch -a
  bugfix/user-auth-error
* collab
  develop
  feature/reject-demo
  main
  staging
  remotes/origin/HEAD -> origin/main
  remotes/origin/bugfix/user-auth-error
  remotes/origin/collab
  remotes/origin/develop
  remotes/origin/feature/reject-demo
  remotes/origin/main
  remotes/origin/staging
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 92: Creating the collab branch and pushing it to origin

17.2 Adding Kiran's Name and Fun Fact

- Appended the name and fun fact line to notes.txt, committed it, and pushed to collab.
- The first version of the line still had a placeholder. It was corrected with a new commit (Fix fun fact line in notes.txt) so that the history stays clear.

```
printf "Name: Kiran Siddique - Fun fact: <fact>\n" > notes.txt
cat notes.txt
git add notes.txt
git commit -m "Fix fun fact line in notes.txt"
git push origin collab
```

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ cd ~/DevOps_Lab2_Practice
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout collab
Already on 'collab'
Your branch is up to date with 'origin/collab'.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git status
On branch collab
Your branch is up to date with 'origin/collab'.

nothing to commit, working tree clean
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ printf "Name: Kiran Siddique - Fun f
act: I enjoy learning new technologies and building small projects\n" > notes.txt

kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ cat notes.txt
Name: Kiran Siddique - Fun fact: I enjoy learning new technologies and building s
mall projects
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git add notes.txt
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Fix fun fact line in
notes.txt"
[collab d5db441] Fix fun fact line in notes.txt
 1 file changed, 1 insertion(+), 4 deletions(-)
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git push origin collab
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 368 bytes | 4.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:KiranSiddique/DevOps_Lab2_Practice.git
   3dbe6d9..d5db441  collab -> collab
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 93: Corrected fun fact line committed and pushed to collab

17.3 Second Account's Contribution

- Logged in as kiransiddique406-source and opened notes.txt on the collab branch in the GitHub web editor.
- Added the second account's name and fun fact on a new line and committed directly to collab.
- On the VM, pulled the latest collab, checked notes.txt, and viewed the commit graph. Both lines and both contributors' commits are visible.

```
git checkout collab
git pull origin collab
cat notes.txt
git log --oneline --graph -5
```

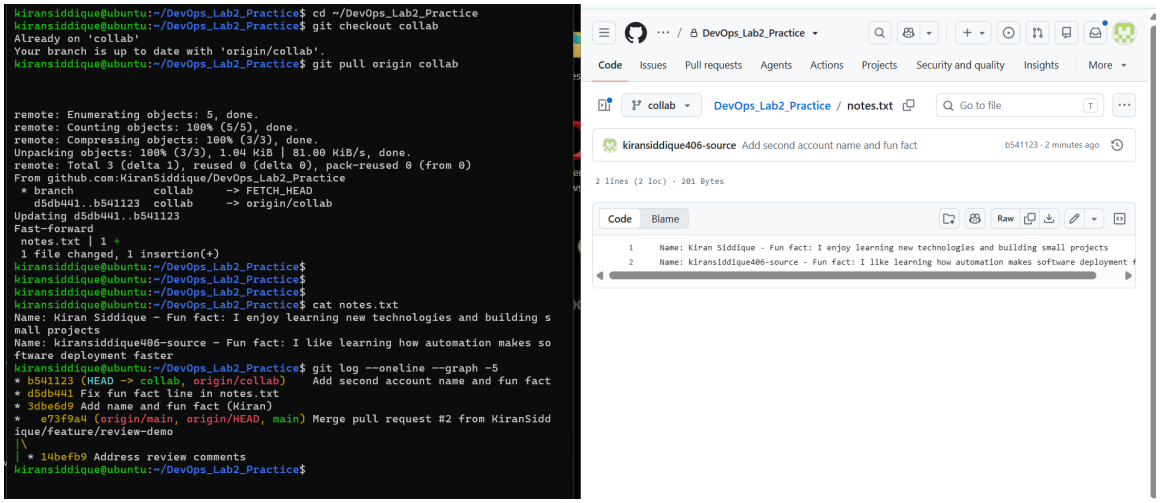


Figure 94: Pulling the second account's commit into collab; both contributions in notes.txt

17.4 Merging collab into main

- Switched to main, pulled the latest changes, and merged collab with a merge commit (--no-ff).
- Pushed main to origin.

```
git checkout main
git pull origin main
git merge --no-ff collab -m "Merge collab into main (bonus collaboratio
git push origin main
```

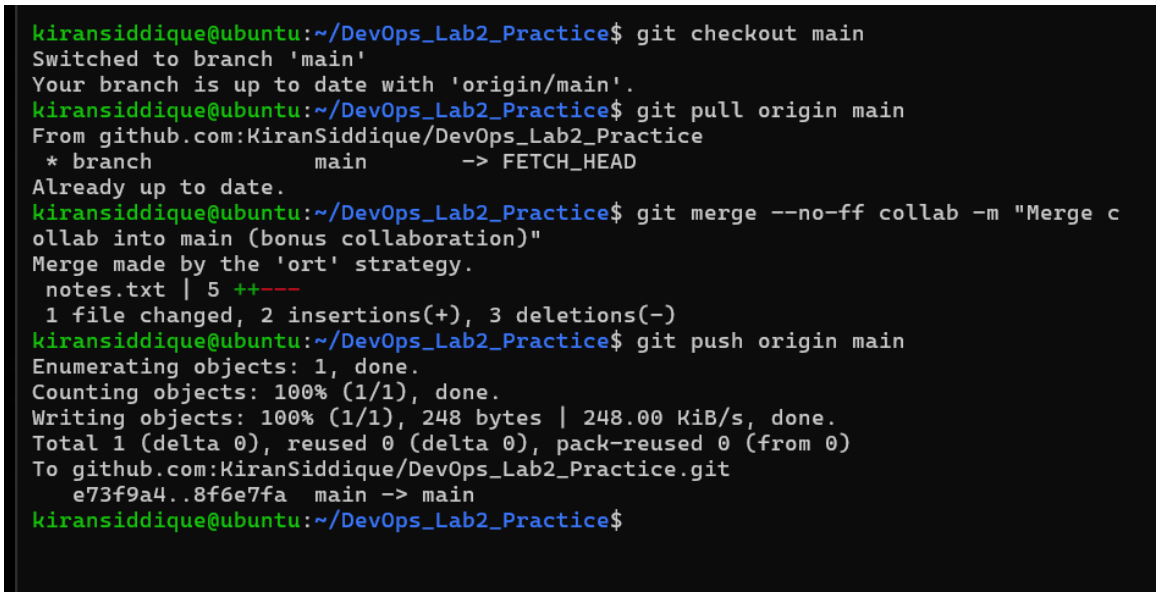


Figure 95: Merging collab into main and pushing

18. Exam Question Q1: Advanced Branching and Merge Verification

18.1 Creating the Branch

A new branch q1-branch was created from main. A branch with the same name already existed from an earlier attempt, so the existing branch was checked out instead of being created again.

```
git checkout main
git pull origin main
git checkout -b q1-branch
git branch
```

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git pull origin main
From github.com:KiranSiddique/DevOps_Lab2_Practice
 * branch            main       -> FETCH_HEAD
Already up to date.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout -b q1-branch
Switched to a new branch 'q1-branch'
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git branch
  bugfix/user-auth-error
  collab
  develop
  feature/reject-demo
  main
* q1-branch
  staging
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 96: Branch q1-branch checked out and listed

18.2 Making a Small Change and Committing

```
echo "Q1: small change for merge verification" >> notes.txt
git add notes.txt
git commit -m "Q1: small change"
git log --oneline -1
```

The commit a31c174 (Q1: small change) was created on q1-branch with one insertion in notes.txt.

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git pull origin main
From github.com:KiranSiddique/DevOps_Lab2_Practice
 * branch            main       -> FETCH_HEAD
Already up to date.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout -b q1-branch
fatal: a branch named 'q1-branch' already exists
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git branch
  bugfix/user-auth-error
  collab
  develop
  feature/reject-demo
* main
  q1-branch
  staging
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 97: Small change committed on q1-branch

18.3 Merging Back into main

```
git checkout main
git merge --no-ff q1-branch -m "Merge q1-branch into main"
git push origin main
```

The merge commit 944f7df (Merge q1-branch into main) was created with --no-ff, so the merge is visible in history. A later re-run of the same command reported Already up to date, which confirms that the merge is complete.

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git merge --no-ff q1-branch -m "Merg
e q1-branch into main"
Merge made by the 'ort' strategy.
 notes.txt | 1 +
 1 file changed, 1 insertion(+)
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git push origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 426 bytes | 142.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 2 local objects.
To github.com:KiranSiddique/DevOps_Lab2_Practice.git
   8f6e7fa..944f7df  main -> main
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 98: Merge of q1-branch into main

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git merge --no-ff q1-branch -m "Merg
e q1-branch into main"
Already up to date.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git push origin main
Everything up-to-date
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 99: Repeat of the merge command showing that main is already up to date

18.4 Verifying the Merge History

```
git log --oneline --graph --all -10
```

The graph shows the merge commit 944f7df with two parents: the q1-branch commit a31c174 and the main commit 8f6e7fa.

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git log --oneline --graph --all -10
*   944f7df (HEAD -> main, origin/main, origin/HEAD) Merge q1-branch into main
|\
| * a31c174 (q1-branch) Q1: small change
|/
*   8f6e7fa Merge collab into main (bonus collaboration)
|\
| * b541123 (origin/collab, collab)    Add second account name and fun fact
| * d5db441 Fix fun fact line in notes.txt
| * 3dbe6d9 Add name and fun fact (Kiran)
|/
*   e73f9a4 Merge pull request #2 from KiranSiddique/feature/review-demo
|\
| * 14befb9 Address review comments
| * dbccefc Add review demo file
|/
| * fc59d5e (origin/feature/reject-demo, feature/reject-demo) Add reject demo fil
e
|/
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 100: Commit graph verifying the merge of q1-branch into main

19. Exam Question Q2: Multi-Stage Workflow Simulation

Three long-lived branches (main, develop and staging) were used. A feature branch was created from develop, merged into develop, then develop was merged into staging, and staging was merged into main. A commit check then confirmed that the feature commit exists in every stage.

19.1 Branches

```
git checkout main && git pull origin main
git checkout develop && git pull origin develop
git checkout staging && git pull origin staging
git branch -a
```

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git pull origin main
From github.com:KiranSiddique/DevOps_Lab2_Practice
 * branch            main       -> FETCH_HEAD
Already up to date.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout develop
Switched to branch 'develop'
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git pull origin develop
From github.com:KiranSiddique/DevOps_Lab2_Practice
 * branch            develop    -> FETCH_HEAD
Already up to date.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout staging
Switched to branch 'staging'
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git pull origin staging

From github.com:KiranSiddique/DevOps_Lab2_Practice
 * branch            staging    -> FETCH_HEAD
Already up to date.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git branch -a
  bugfix/user-auth-error
  collab
  develop
  feature/reject-demo
  main
  q1-branch
* staging
  remotes/origin/HEAD -> origin/main
  remotes/origin/bugfix/user-auth-error
  remotes/origin/collab
  remotes/origin/develop
  remotes/origin/feature/reject-demo
  remotes/origin/main
  remotes/origin/staging
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 101: The main, develop and staging branches, local and remote

19.2 Feature Branch and Commit

```
git checkout develop
git checkout -b feature/q2 develop
echo "Q2 feature work" > q2.txt
git add q2.txt
git commit -m "Q2: feature commit"
git log --oneline -2
```

The commit 351371f (Q2: feature commit) is on feature/q2. The log also shows the merge commit 8c4993c, which was created when the feature was merged into develop.

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout develop
Already on 'develop'
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git pull origin develop
From github.com:KiranSiddique/DevOps_Lab2_Practice
 * branch            develop    -> FETCH_HEAD
Already up to date.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout -b feature/q2 develop
fatal: a branch named 'feature/q2' already exists
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ echo "Q2 feature work" > q2.txt
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git add q2.txt
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Q2: feature commit"
On branch develop
nothing to commit, working tree clean
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git log --oneline -2
8c4993c (HEAD -> develop, origin/develop) Merge feature/q2 into develop
351371f (feature/q2) Q2: feature commit
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 102: Feature branch feature/q2 with its commit in the log

19.3 Merging into develop

```
git checkout develop
git merge --no-ff feature/q2 -m "Merge feature/q2 into develop"
git push origin develop
```

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout develop
Already on 'develop'
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git merge --no-ff feature/q2 -m "Mer
ge feature/q2 into develop"
Already up to date.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git push origin develop
Everything up-to-date
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 103: Merge of feature/q2 into develop

19.4 Merging develop into staging

```
git checkout staging
git pull origin staging
git merge --no-ff develop -m "Merge develop into staging"
git push origin staging
```

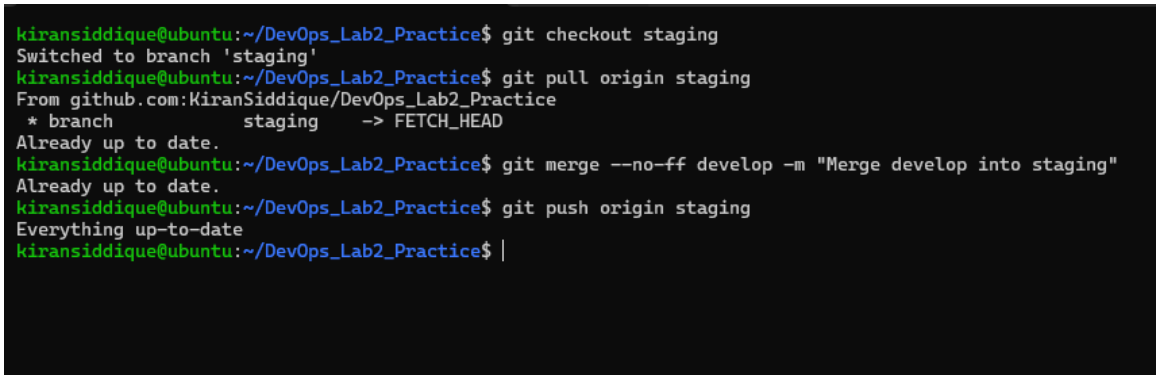


Figure 104: Merge of develop into staging (staging already up to date)

19.5 Merging staging into main

```
git checkout main
git pull origin main
git merge --no-ff staging -m "Merge staging into main"
git push origin main
```

When the staging merge was run again, it produced a real merge commit that brings the q2.txt change into main.

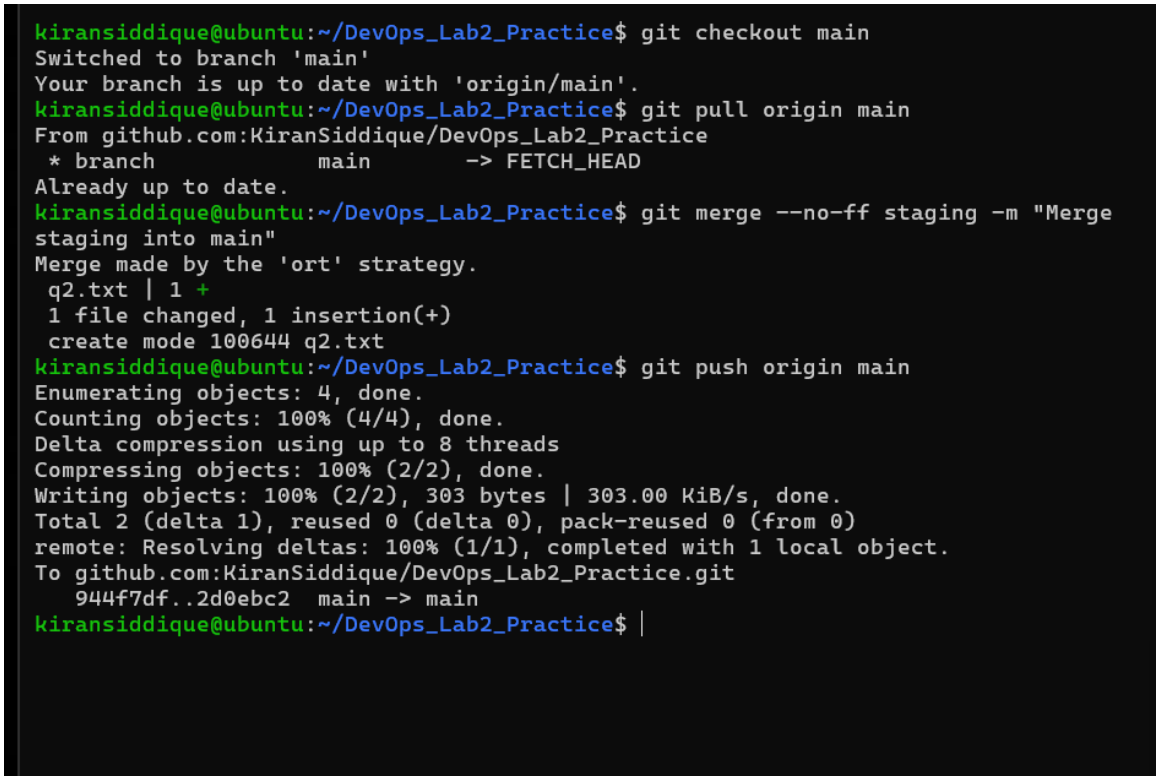


Figure 105: Merge of staging into main, adding q2.txt to main

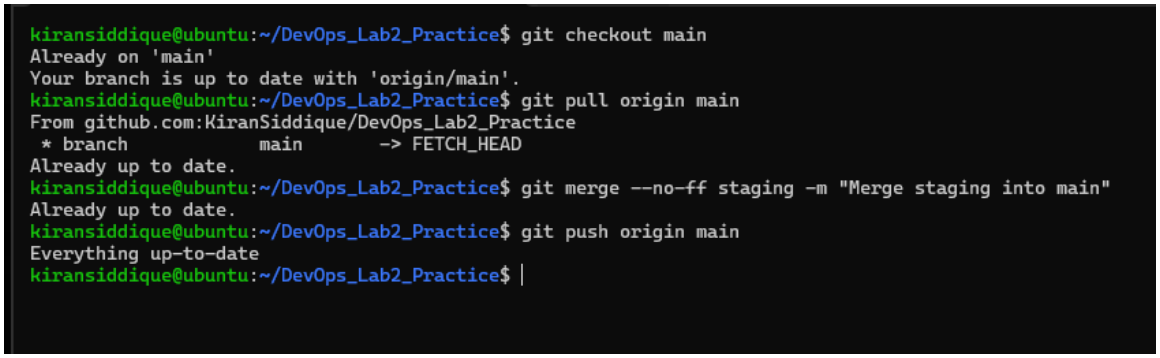


Figure 106: Merge of staging into main and push to origin

19.6 Stage Verification

```
git branch -a --contains 351371f
```

The output lists develop, staging and main (and their remote branches) as containing the feature commit. This proves that the change passed through every stage before it reached main.



Figure 107: Branches that contain the feature commit 351371f

20. Exam Question Q3: Collaboration and Conflict Resolution

Two contributors changed the same line of shared.txt in separate branches. The merge produced a conflict, which was resolved so that both contributions remain.

20.1 Modifying the Shared File in Two Branches

```
git checkout main
git pull origin main
echo "Shared base line" > shared.txt
git add shared.txt
git commit -m "Add shared.txt base"
git push origin main
git checkout -b q3-kiran
echo "Shared line: Kiran's version" > shared.txt
git add shared.txt
git commit -m "Q3: Kiran change"
git push -u origin q3-kiran
```

The second account created q3-second from main in the GitHub web editor and changed shared.txt there, committing directly to that branch.

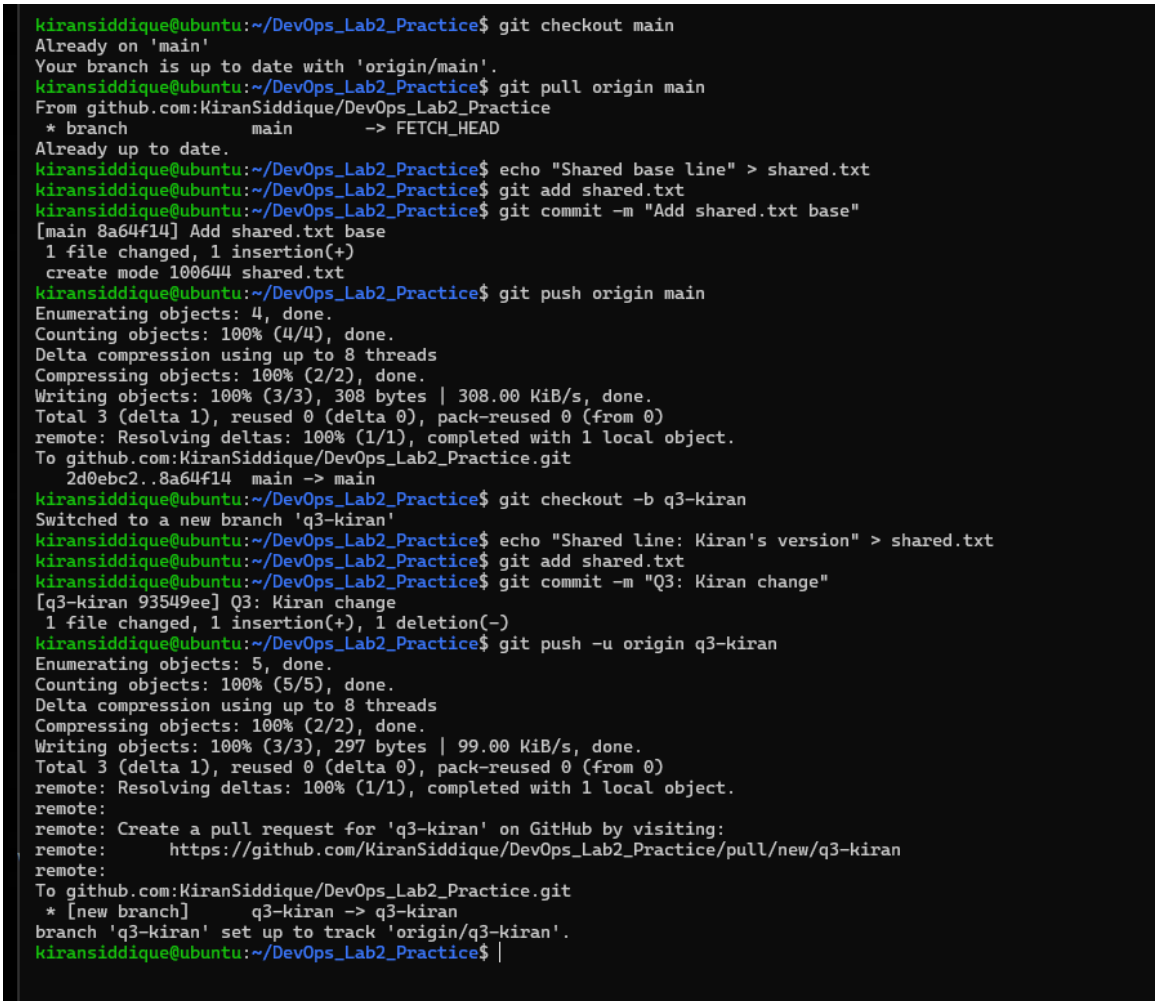


Figure 108: Kiran's branch q3-kiran pushed with its change

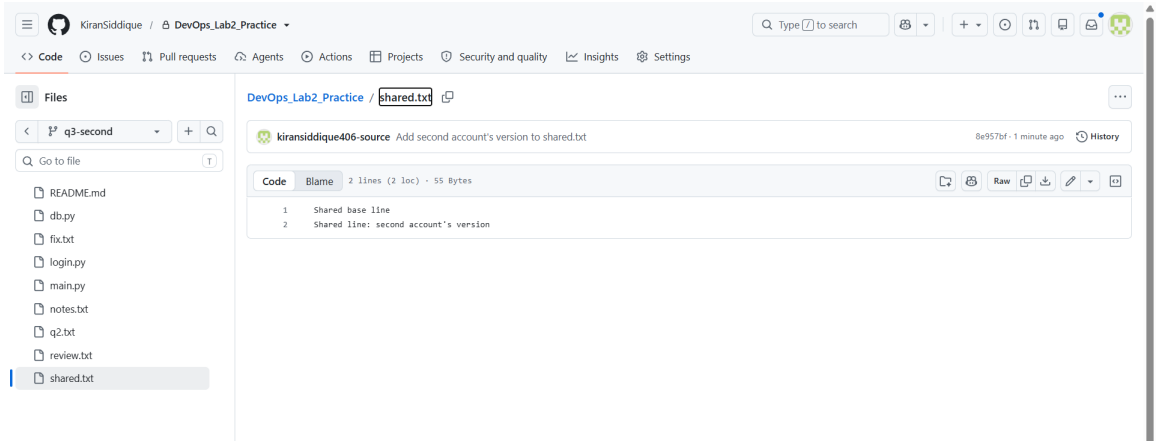


Figure 109: Second account's change committed to q3-second



Figure 110: Branches page showing q3-kiran and q3-second

20.2 Merging and the Conflict

```
git checkout main
git merge q3-kiran
git fetch origin
git merge origin/q3-second
```

The first merge was a fast-forward. The second merge reported a conflict in shared.txt, because both branches changed the same line.

```
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git pull origin main
From github.com:KiranSiddique/DevOps_Lab2_Practice
 * branch            main       -> FETCH_HEAD
Already up to date.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git merge q3-kiran
Updating 8a64f14..93549ee
Fast-forward
 shared.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git fetch origin
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 972 bytes | 162.00 KiB/s, done.
From github.com:KiranSiddique/DevOps_Lab2_Practice
 * [new branch]      q3-second  -> origin/q3-second
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$ git merge origin/q3-second
Auto-merging shared.txt
CONFLICT (content): Merge conflict in shared.txt
Automatic merge failed; fix conflicts and then commit the result.
kiransiddique@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 111: Conflict in shared.txt during the merge

20.3 Resolving the Conflict

```
nano shared.txt
git add shared.txt
git commit -m "Resolve conflict: keep both contributions"
git push origin main
```

In nano, the conflict markers were removed so that both lines remain. The resolved commit was pushed to main.

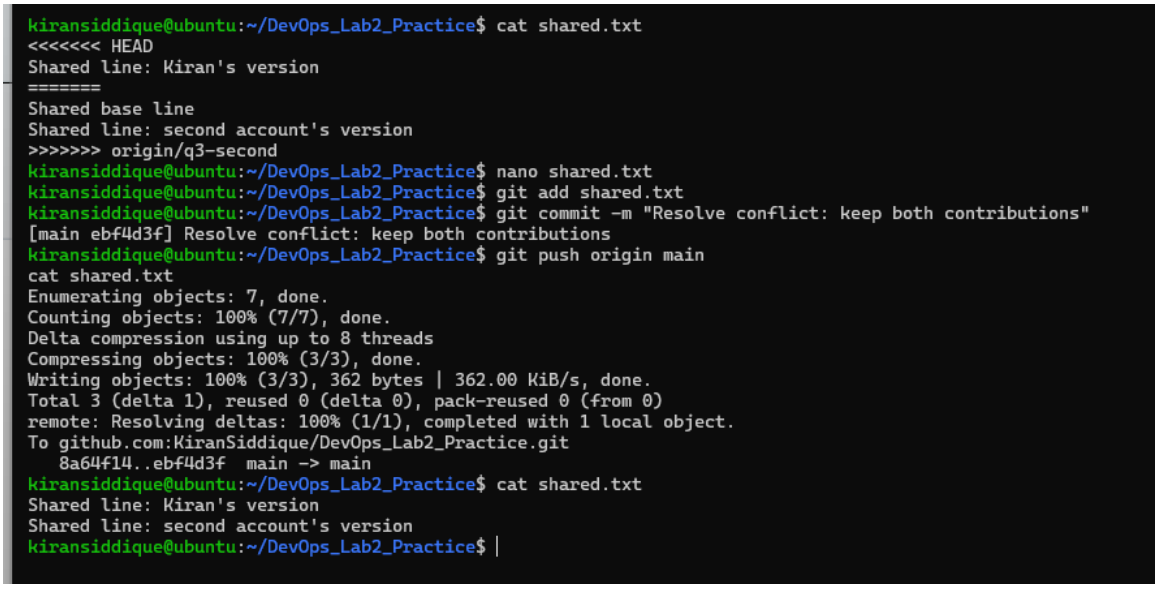


Figure 112: Conflict resolved and pushed to main

20.4 Final File

```
cat shared.txt
```

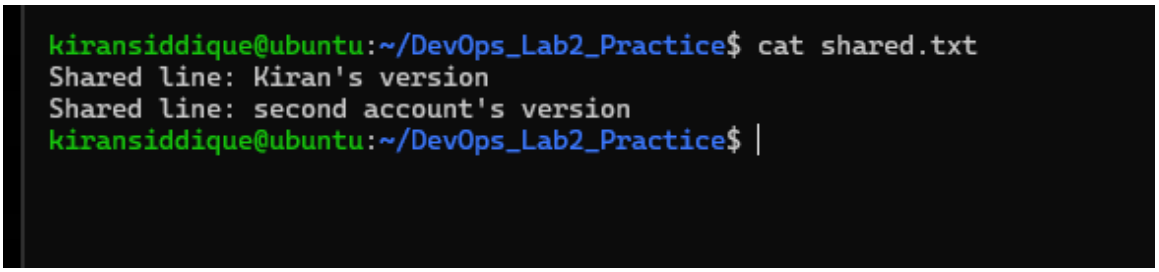


Figure 113: Final shared.txt containing both contributions

21. Submission Structure

The final evidence was placed in the DevOps repository under Labs/Lab02, using the file names required by the lab. The layout is as follows.

```
DevOps/
  Labs/
    Lab02/
      README.md
      notes.txt
      main.py
      Lab2_Solution.pdf
      screenshots/   (all required screenshots)
```

Commands used for the submission:

```
cd ~/DevOps
git status
git add Labs/Lab02
git commit -m "Submit Lab 02 evidence"
git push origin main
```